

SET

▼ Defining Software

Software is more than just programs. It consists of

1. **Instructions (programs)** that provide required features, functions, and performance
2. **Data structures** that allow information to be manipulated
3. **Documentation** that explains how the software operates and is used

Unlike physical systems, **software is a logical system**, and this makes it fundamentally different from hardware.

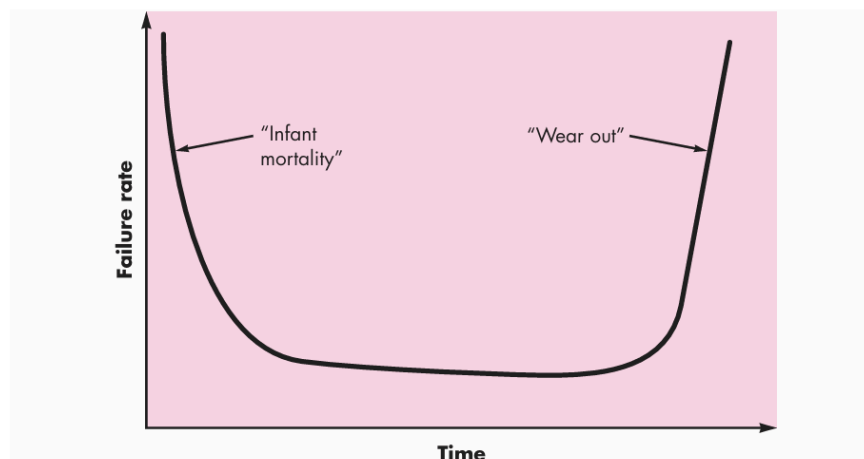
Key Characteristics of Software

1. Software is engineered, not manufactured

Software development resembles engineering and design rather than manufacturing.

- Quality depends mainly on **good design**, not production.
- Software costs are concentrated in the **development phase**, not replication.
- Software projects cannot be managed like manufacturing projects.

FIGURE 1.1
Failure curve
for hardware

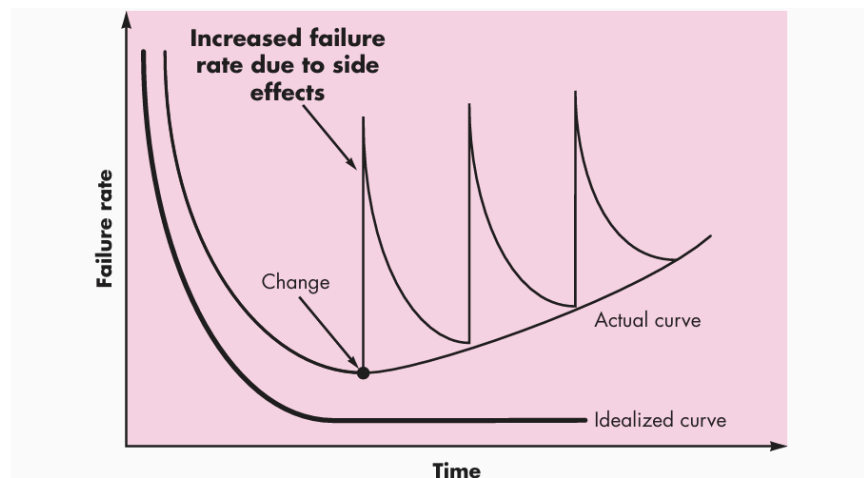


2. Software does not wear out, but it deteriorates

- Hardware follows a *bathtub curve* with early failures, stable operation, and eventual wear-out.
- Software does **not degrade due to physical aging**.
- However, software **deteriorates over time** because frequent changes introduce new errors.
- Each modification can increase the failure rate, causing long-term quality decline.
- Unlike hardware, there are **no spare parts**—every failure reflects a design or implementation error.

FIGURE 1.2

Failure curves for software



3. Most software is still custom-built

- Other engineering fields rely heavily on standardized, reusable components.
- Software engineering is moving toward **component-based development**, but reuse is still limited.
- Modern software components encapsulate both **data and behavior**, enabling reuse (e.g., GUI libraries).
- Greater reuse allows engineers to focus on innovation rather than rebuilding common functionality.

Overall Insight

Software's intangible nature, susceptibility to deterioration through change, and limited reuse make it uniquely challenging to build and maintain. These challenges highlight the **need for disciplined software engineering practices**.

▼ Infant Mortality ?

Infant Mortality (in hardware)

It refers to the **high failure rate at the very beginning of a system's life**.

- ♦ These early failures happen **soon after deployment**
- ♦ Causes include:
 - Design flaws
 - Manufacturing defects
 - Faulty components
 - Assembly or installation errors

Once these initial defects are **detected and fixed**, the failure rate drops and the hardware enters a stable phase.

▼ Software Application Domains

Today, seven broad categories of computer software present continuing challenges for software engineers:

System software—a collection of programs written to service other programs. Some system software (e.g., compilers, editors, and file management utilities) processes complex, but determinate,⁴ information structures. Other systems applications (e.g., operating system components, drivers, networking software, telecommunications processors) process largely indeterminate data. In either case, the systems software area is characterized by heavy interaction with computer hardware; heavy usage by multiple users; concurrent operation that requires scheduling, resource sharing, and sophisticated process management; complex data structures; and multiple external interfaces.

Application software—stand-alone programs that solve a specific business need. Applications in this area process business or technical data in a way that facilitates business operations or management/technical decision making. In addition to conventional data processing applications, application software is used to control business functions in real time (e.g., point-of-sale transaction processing, real-time manufacturing process control).

Engineering/scientific software—has been characterized by “number crunching” algorithms. Applications range from astronomy to volcanology, from automotive stress analysis to space shuttle orbital dynamics, and from molecular biology to automated manufacturing. However, modern applications within the engineering/scientific area are moving away from

conventional numerical algorithms. Computer-aided design, system simulation, and other interactive applications have begun to take on real-time and even system software characteristics.

Embedded software—resides within a product or system and is used to implement and control features and functions for the end user and for the system itself. Embedded software can perform limited and esoteric functions (e.g., key pad control for a microwave oven) or provide significant function and control capability (e.g., digital functions in an automobile such as fuel control, dashboard displays, and braking systems).

Product-line software—designed to provide a specific capability for use by many different customers. Product-line software can focus on a limited and esoteric marketplace (e.g., inventory control products) or address mass consumer markets (e.g., word processing, spreadsheets, computer graphics, multimedia, entertainment, database management, and personal and business financial applications).

Web applications—called “WebApps,” this network-centric software category spans a wide array of applications. In their simplest form, WebApps can be little more than a set of linked hypertext files that present information using text and limited graphics. However, as Web 2.0 emerges, WebApps are evolving into sophisticated computing environments that not only provide stand-alone features, computing functions, and content to the end user, but also are integrated with corporate databases and business applications.

Artificial intelligence software—makes use of nonnumerical algorithms to solve complex problems that are not amenable to computation or straightforward analysis. Applications within this area include robotics, expert systems, pattern recognition (image and voice), artificial neural networks, theorem proving, and game playing.

▼ IEEE Definition of Software Engineering

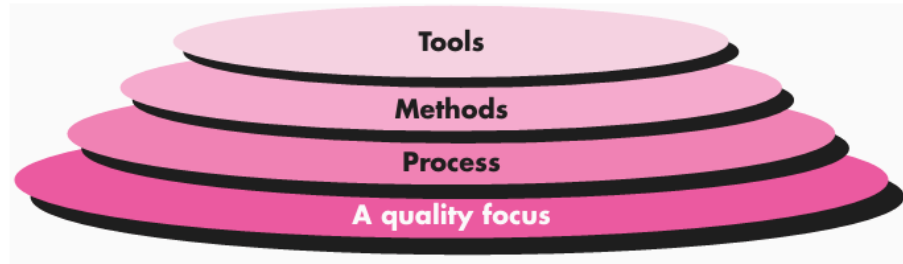
S/w engineering is a systematic , disciplined and quantifiable approach to develop and maintain a software.

▼ Software Engineering Layers

Software engineering is a **layered technology** in which each layer builds upon the one below it to ensure the development of high-quality, reliable, and maintainable software. The layers are **Quality Focus, Process, Methods, and Tools**.

FIGURE 1.3

Software
engineering
layers



1. Quality Focus

The foundation of software engineering is a strong **commitment to quality**. It represents the organizational culture that emphasizes doing work correctly from the beginning. Quality is not added at the end but is built into every activity. Practices such as continuous improvement and defect prevention are part of this layer.

2. Process Layer

The process layer forms the **framework** for software development. It defines the sequence of activities, milestones, deliverables, and controls required to manage a software project. The process ensures timely development, proper change management, and consistent monitoring of progress and quality.

A simple software process might be:

1. Understand the problem
2. Plan the solution
3. Design it
4. Write the code
5. Test it
6. Deliver it
7. Maintain it

3. Methods Layer

Software engineering methods provide the **technical procedures** for building software. These include techniques for requirements analysis, system design, coding, testing, and maintenance. Methods specify *how* each task in the process is performed and help convert user requirements into a working software product.

Methods are the **technical techniques**, such as:

- How to collect requirements from users
- How to design the software structure
- How to write clean, correct code
- How to test software properly

4. Tools Layer

Tools provide **automated or semi-automated support** for the process and methods. Examples include compilers, debuggers, testing tools, and modeling tools. When tools are integrated, they form Computer-Aided Software Engineering (CASE) environments that improve productivity and consistency.

Examples of tools:

- Code editors (VS Code)
- Compilers
- Debuggers
- Testing tools
- Version control (Git)

Tools:

- Do not replace thinking
- Only **support** the process and methods

Conclusion

All four layers work together to produce high-quality software. A strong quality focus supports the process, the process guides the methods, and tools enhance the efficiency of methods.

▼ Software Process Framework

A **software process** is a collection of **activities, actions, and tasks** performed to create a software product.

- An **activity** achieves a broad objective (e.g., communication with stakeholders).
- An **action** consists of related tasks that produce a major work product (e.g., architectural design).
- A **task** is a small, well-defined unit of work that produces a tangible result (e.g., unit testing).

A software process is **not rigid**; it is **adaptable and flexible**, allowing teams to choose appropriate activities and tasks to deliver software **on time and with acceptable quality**.

Generic Software Process Framework Activities

All software projects, regardless of size or complexity, follow five generic framework activities:

1. **Communication**

Involves interaction with customers and stakeholders to understand objectives and gather requirements.

2. **Planning**

Establishes a project plan that defines tasks, risks, resources, schedules, and work products.

3. **Modeling**

Involves creating models and designs to understand requirements and define the software structure.

4. **Construction**

Combines coding and testing to build the software and detect errors.

5. **Deployment**

Delivers the software to customers, who evaluate it and provide feedback.

These activities are applied **iteratively**, with each iteration producing a working **software increment**.

Umbrella Activities

Umbrella activities are **supporting activities** that are carried out **throughout the entire software development process**.

They **do not belong to just one phase** like design or coding.

Instead, they **span across all framework activities** (communication, planning, modeling, construction, deployment).

Umbrella activities are performed **throughout the entire software process** to manage and control the project. They include:

- Software project tracking and control
- Risk management
- Software quality assurance
- Technical reviews
- Measurement
- Software configuration management
- Reusability management
- Work product preparation and documentation

These activities help ensure **quality, consistency, and controlled change**.

Process Adaptation and Models

The software process must be **adapted** based on:

- Project size and complexity
- Stakeholder involvement
- Required rigor and documentation
- Team autonomy and organization

Process Models

- **Prescriptive process models** emphasize detailed planning and control but can become bureaucratic if applied rigidly.
- **Agile process models** emphasize flexibility, adaptability, and customer collaboration, making them suitable for dynamic projects such as web applications.

▼ Prescriptive Process Models

Q1. What are Prescriptive Process Models? Explain.

Prescriptive process models are traditional software process models that define a **structured and disciplined approach** to software development. They prescribe a **set of framework activities, tasks, work products, quality assurance steps, and change control mechanisms**, along with a **predictable workflow**.

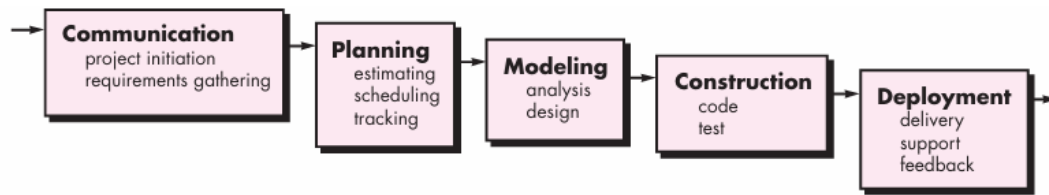
These models were introduced to bring **order and consistency** to software development, which was earlier chaotic and unorganized. They emphasize **detailed planning, documentation, and control** to ensure predictable results.

However, software development often exists at the **edge of chaos**, where too much rigidity reduces flexibility and innovation, while too little structure leads to disorder. Therefore, prescriptive models are useful when requirements are **well-defined and stable**, but may become inefficient in rapidly changing environments.

Q2. Explain the Waterfall Model.

The **Waterfall Model** is the oldest and most widely known prescriptive process model. It follows a **linear and sequential approach**, where each phase must be completed before the next phase begins.

FIGURE 2.3 The waterfall model



Phases of the Waterfall Model

1. Communication (requirements gathering)
2. Planning
3. Modeling (analysis and design)
4. Construction (coding and testing)
5. Deployment (delivery and maintenance)

Advantages

- Simple and easy to understand
- Clear documentation and milestones
- Suitable for small projects with stable requirements

Disadvantages

- Difficult to accommodate changing requirements
- Working software is delivered very late
- Errors discovered late are costly to fix
- Real projects rarely follow a strict sequence

Prototyping Process Model

The **Prototyping Process Model** is used when software requirements are not clearly defined or are likely to change. Instead of developing the complete system at once, a **working prototype** is built quickly to help users understand

the system. Users evaluate the prototype and give feedback, which is used to refine requirements through multiple iterations. The prototype helps both developers and users gain a clearer understanding of what the final system should be.

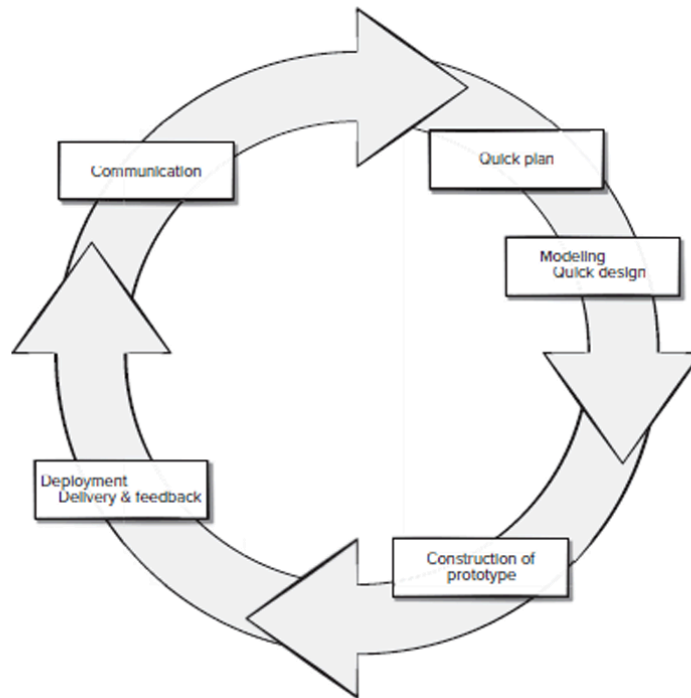
Advantages

- Helps clarify unclear or incomplete requirements
- Users can see and interact with the system early
- Reduces misunderstandings between users and developers
- Improves user satisfaction through continuous feedback

Disadvantages

- Users may assume the prototype is the final system
- Developers may make quick design shortcuts that affect quality
- Poor documentation may result
- Not suitable for large, highly complex systems

FIGURE 2.4 The prototyping paradigm



Evolutionary Process Model (Spiral Model)

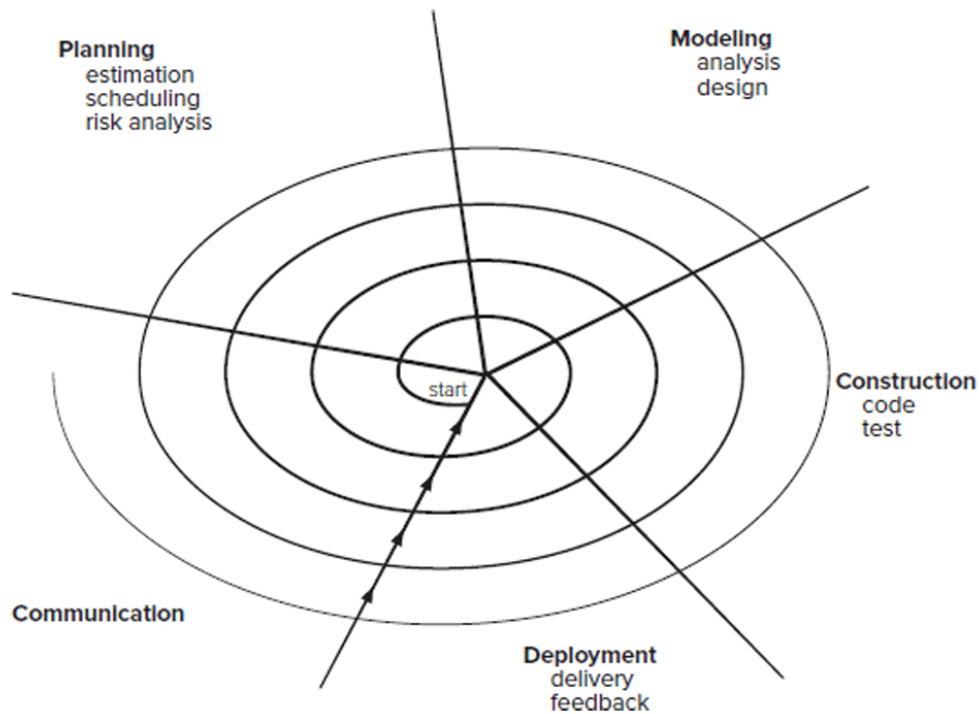
The **Evolutionary Process Model** develops software in a series of **incremental and iterative releases**. The system evolves over time as new features are added based on customer feedback. The **Spiral Model** combines iterative development with systematic planning and risk analysis. Each cycle of the spiral includes planning, risk assessment, development, and evaluation, allowing the product to improve gradually.

Advantages

- Accommodates changing requirements easily
- Early delivery of working software
- Continuous customer feedback improves quality
- Risk analysis helps prevent major failures

Disadvantages

- Requires experienced developers for risk management
- Can be costly and time-consuming
- Difficult to manage for small projects
- Customers may find it hard to understand the process

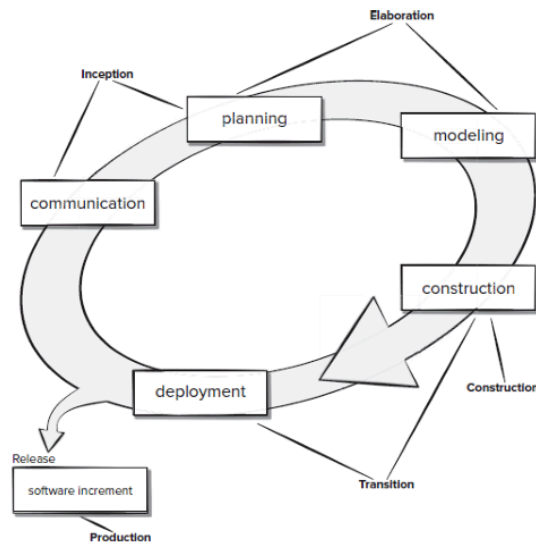


Q7. Why can Prescriptive Process Models become Bureaucratic?

Prescriptive process models can become **bureaucratic** when applied too rigidly. Excessive documentation, approvals, and formal procedures can slow development, reduce creativity, and resist change. Instead of supporting development, the process itself becomes the focus, leading to inefficiency and frustration among developers.

Unified Process (UP) Model

The Unified Process is an **iterative and incremental software process model** that combines the strengths of traditional models with **agile principles**. It emphasizes **customer communication, use-case-driven development**, and a **strong software architecture**. UP uses **UML** for modeling and supports evolutionary development through repeated software increments.



Phases of the Unified Process

1. Inception Phase

Focuses on customer communication and planning. Basic business requirements are identified using **preliminary use cases**, major risks are assessed, and a rough project plan and schedule are created.

2. Elaboration Phase

Requirements are refined and expanded. A solid **architectural baseline** is developed using multiple views such as use-case, analysis, design, implementation, and deployment models. Project plans are refined and risks are further reduced.

3. Construction Phase

Actual coding and testing take place. Software components are implemented, unit and integration testing are performed, and use cases are used to derive acceptance tests for each increment.

4. Transition Phase

The software is delivered to users for **beta testing**. Feedback is collected, defects are corrected, and final adjustments are made before releasing a usable version of the software.

5. Production Phase

The software is deployed and maintained. System performance is monitored, user support is provided, and change requests and defect reports are managed.

▼ Software Measurement

SE 38: Software Measurements & Metrics | LOC | FP with Examples #softwareengineering

Software Measurements

- Software Measurement is indicator of size, quantity amount or dimension of particular attribute of a product or process.
- It helps the project manager & entire software team to take decisions that lead to successful completion of the project by generating quantity result.

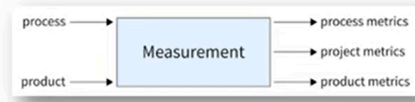
➤ Software measurements are of two categories:

1. **Direct Measures:** It include software processes like Cost & Effort applied, Lines of code produced, Execution speed & Total no. of errors that have been reported.
2. **Indirect Measures:** It include products like Functionality, Quality, Complexity, Reliability, Maintainability and many more.



Software Metrics

- Software Metrics provide measures, functions or formulas for various aspects of software process & product.
- It including measuring software performance, planning work items, measuring productivity, and many other uses.



➤ Software metrics are of three categories:

1. **Product Metrics:** It estimate Size, Complexity, Quality & Reliability of software.
2. **Process Metrics:** It estimate Faults rate during development, Pattern of testing defect arrival, Time it takes for a fixed operation.
3. **Project Metrics:** It estimate Number of software developers, Cost, Scheduling and Productivity of software.

Software Measurement Principles

1. **Formulation:** The derivation of software measures and metrics appropriate for the representation of the software that is being considered.
2. **Collection:** The mechanism used to accumulate data required to derive the formulated metrics.
3. **Analysis:** The computation of metrics and the application of mathematical tools.
4. **Interpretation:** The evaluation of metrics resulting in insight into the quality of the representation.
5. **Feedback:** Recommendation derived from the interpretation of product metrics transmitted to the software team.

▼ Requirements model

What is a Requirements Model?

A **requirements model** is a **structured representation of what the software system must do**, describing **functional and non-functional requirements** in a way that is easy to analyze, validate, and use as a basis for design.

👉 In short:

It explains *what* the system should do, not *how* it will be built.

Purpose of the Requirements Model

The requirements model is created to:

- Understand **user needs**
 - Clearly define **system behavior**
 - Serve as a **foundation for design**
 - Reduce ambiguity and misunderstandings
 - Enable **early validation** and **estimation**
-

What Does the Requirements Model Contain?

It typically includes:

1 Scenario-Based Elements

- Use cases
 - User stories
 - Interaction flows
- 📌 Show how users interact with the system
-

2 Data Model

- Data objects
 - Attributes
 - Relationships
- 📌 Defines the data the system uses and manages
-

3 Functional Model

- Processes
 - Transformations
 - Data flow diagrams (DFDs)
 - 📌 Shows how input is transformed into output
-

4 Behavioral Model

- States
 - Events
 - State transition diagrams
 - 📌 Describes how the system behaves over time
-

5 Non-Functional Requirements

- Performance
 - Security
 - Reliability
 - Usability
 - 📌 Constraints and quality attributes
-

Why is the Requirements Model Important?

- Forms the **basis for architecture and design**
 - Enables **product metrics** (like Function Points)
 - Helps estimate **size, effort, and cost**
 - Supports **test case derivation**
 - Detects missing or inconsistent requirements early
-

Example (Simple)

For an **ATM system**, the requirements model may include:

- Use case: *Withdraw Cash*
- Data objects: *Account, Transaction*
- Functional flow: *Validate PIN → Check balance → Dispense cash*
- Behavioral states: *Idle, Processing, Error*

▼ Project estimate

- **Product metrics** at the requirements stage help assess **analysis model quality**.
 - Since design and code don't exist yet, we estimate **software size** from requirements.
 - **Size ≠ quality**, but larger size usually means:
 - Higher complexity
 - More coding effort
 - More testing and integration work
-

Function-Based Metrics (Function Points – FP)

What are Function Points?

- **Function Points (FP)** measure the **functionality delivered to the user**, not lines of code.
 - FP is **language-independent** and can be computed **early** from requirements.
-

Why use Function Points?

Using historical data, FP helps to:

1. Estimate **cost and development effort**
 2. Predict **number of defects**
 3. Forecast **code size (LOC)** and number of components
-

Information Domain Values (What we count)

Function Points are calculated by counting five things in the requirements model:

1 External Inputs (EI)

- Inputs from users or other systems
 - Used to update internal data
-  Example: login data, button presses

2 External Outputs (EO)

- Data sent to users
-  Example: reports, messages, alerts

3 External Inquiries (EQ)

- Input that immediately produces an output
-  Example: "check sensor status"

4 Internal Logical Files (ILF)

- Logical data stored **inside** the system
-  Example: configuration files, databases

5 External Interface Files (EIF)

- Data used by the system but stored **outside** it
-  Example: shared databases, external systems
-

Weighting by Complexity

Each item is classified as:

- **Simple**
- **Average**
- **Complex**

Each category has a **predefined weight**.

Weighted counts are added to get:

👉 **Unadjusted Function Point Count (UFP)**

FIGURE 23.1

Computing
function points

Information Domain Value	Count		Weighting factor				
			Simple	Average	Complex		
External Inputs (EIs)		×	3	4	6	=	
External Outputs (EOs)		×	4	5	7	=	
External Inquiries (EQs)		×	3	4	6	=	
Internal Logical Files (ILFs)		×	7	10	15	=	
External Interface Files (EIFs)		×	5	7	10	=	
Count total							

Value Adjustment Factor (VAF)

To account for system complexity, 14 questions are answered, such as:

- Is performance critical?
- Is distributed processing used?
- Is online data entry required?
- Is the system designed for change?

Each question is rated **0 to 5**.

The F_i ($i = 1$ to 14) are *value adjustment factors* (VAF) based on responses to the following questions [Lon02]:

1. Does the system require reliable backup and recovery?
2. Are specialized data communications required to transfer information to or from the application?
3. Are there distributed processing functions?
4. Is performance critical?
5. Will the system run in an existing, heavily utilized operational environment?
6. Does the system require online data entry?
7. Does the online data entry require the input transaction to be built over multiple screens or operations?
8. Are the ILFs updated online?
9. Are the inputs, outputs, files, or inquiries complex?
10. Is the internal processing complex?
11. Is the code designed to be reusable?
12. Are conversion and installation included in the design?
13. Is the system designed for multiple installations in different organizations?
14. Is the application designed to facilitate change and ease of use by the user?

Each of these questions is answered using a scale that ranges from 0 (not important or applicable) to 5 (absolutely essential). The constant values in Equation (23.1) and the weighting factors that are applied to information domain counts are determined empirically.

Function Point Formula

$$FP = UFP \times [0.65 + 0.01 \times \sum F_i]$$

Where:

- UFP = total weighted count
- F_i = value adjustment factors (14 questions)

SafeHome Example (Explained Simply)

From the data flow diagram:

- 3 External Inputs
- 2 External Outputs
- 2 External Inquiries
- 1 Internal Logical File
- 4 External Interface Files

After applying weights:

- **UFP = 50**
- Sum of $F_i = 46$ (moderate complexity)

$$FP = 50 \times [0.65 + (0.01 \times 46)] = 56 \quad FP = 50 \times [0.65 + (0.01 \times 46)] = 56$$

$$FP = 50 \times [0.65 + (0.01 \times 46)] = 56$$

How FP Helps in Planning

Using past project data:

- **1 FP = 60 LOC**
- **12 FP = 1 person-month**
- **3 defects/FP** in reviews
- **4 defects/FP** in testing

So FP allows:

- Estimating **code size**
- Estimating **effort**
- Predicting **defects**
- Assessing **review and testing completeness**

All this is done **before coding starts**.

▼ LOC

LOC-Based Metrics (Line of Code Metrics)

What is LOC?

LOC (Lines of Code) is a **size-oriented product metric** that measures software size by **counting the number of lines in the source code**.

👉 It is one of the **oldest and most widely used metrics** in software engineering.

How LOC-Based Metrics Work

Once the total number of lines of code is known or estimated, **other metrics are derived from LOC**, such as productivity, quality, and cost.

These metrics are usually expressed as **ratios**.

Common LOC-Based Metrics

1 Productivity Metric

Measures how much code is produced per unit effort.

$$\text{Productivity} = \frac{\text{LOC}}{\text{Person-Month}}$$

✦ Example:

If 6000 LOC are written in 10 person-months:

Productivity = **600 LOC/person-month**

2 Quality Metric (Defect Density)

Measures software quality in terms of defects.

$$\text{Defect Density} = \frac{\text{Number of Defects}}{\text{KLOC}}$$

✦ Example:

If 30 defects are found in 10,000 LOC:

Defect density = **3 defects/KLOC**

3 Cost Metric

Measures cost per line of code.

$$\text{Cost per LOC} = \frac{\text{Total Project Cost}}{\text{LOC}}$$

✦ Helps estimate budget for future projects.

4 Documentation Metric

Measures documentation quality relative to code size.

$$\text{Pages of Documentation per KLOC}$$

5 Error Rate Metric

Tracks errors found during development or testing.

$$\text{Errors per KLOC}$$

Advantages of LOC-Based Metrics

- ✓ Simple and easy to compute
- ✓ Intuitive and widely understood
- ✓ Useful for effort and defect prediction
- ✓ Works well for **historical trend analysis**

Limitations of LOC-Based Metrics ✗

- ✗ Cannot be used early (before coding)
- ✗ Language-dependent (C ≠ Java ≠ Python)
- ✗ Encourages writing more code
- ✗ Penalizes concise, efficient coding
- ✗ Difficult to define (comments? blank lines?)

▼ Halstead's software metric

1 What is Halstead's Software Science?

Halstead proposed that **software can be measured mathematically**, just like physical systems.

He introduced **analytical laws** based on **operators and operands** used in a program.

👉 These metrics are:

- Derived **after coding**, or
 - **Estimated after design** (before full code exists)
-

2 Primitive Measures (Basic Counts)

Halstead starts with **four fundamental counts**:

◆ n_1 — Distinct operators

- Unique operators used in the program

Examples: `+`, `-`, `if`, `while`, `=`, `return`

◆ n_2 — Distinct operands

- Unique operands (variables, constants)

Examples: `x`, `y`, `count`, `10`

◆ N_1 — Total operator occurrences

- Total number of times operators appear

◆ N_2 — Total operand occurrences

- Total number of times operands appear

📌 These are **direct counts** from source code.

3 Derived Measures

Using the four primitive measures, Halstead defines several important metrics.

♦ Program Length (N)

Total size of the program in terms of tokens:

$$N = N_1 + N_2$$

Estimated length (before code is complete):

$$N = n_1 \log_2 n_1 + n_2 \log_2 n_2$$

♦ Program Vocabulary (n)

Total unique symbols used:

$$n = n_1 + n_2$$

♦ Program Volume (V)

Measures **information content** of the program (in bits):

$$V = N \log_2 (n_1 + n_2)$$

✦ Interpretation:

- Higher volume → more complex program
- Depends on **programming language**

🔢 Program Level (L)

Measures **how compact or efficient** the program is.

$$L = \frac{2}{n_1} \times \frac{n_2}{N_2}$$

- **L < 1 always**
- Higher L → simpler, more readable code
- Lower L → more complex code

Free offer ×

5 Effort Metric (e)

Represents **mental effort** required to understand or test the program.

$$e = \frac{V}{PL}$$

Where:

- V = program volume
- PL = program level

✦ Higher effort → more testing and review required

6 Halstead Metrics in Testing

Halstead metrics are used to:

- Estimate **testing effort**
- Identify **error-prone modules**
- Allocate testing resources

♦ **Testing effort per module**

$$\text{Testing effort percentage for module } k = \frac{e(k)}{\sum e(i)}$$

Meaning:

- Modules with **higher Halstead effort** get **more testing**
- Helps prioritize unit testing

7 Why Halstead Metrics Matter

✓ Advantages

- Based on **formal mathematical theory**
- Language-independent in concept
- Can estimate:
 - Effort
 - Time
 - Faults
 - Testing focus

✗ Limitations

- Controversial assumptions

- Sensitive to counting rules
- Doesn't fully capture logical complexity

▼ Cyclomatic complexity

Cyclomatic Complexity, proposed by **Thomas McCabe**, is a **software metric** used to measure the **logical complexity** of a program. It represents the **number of linearly independent execution paths** in the source code.

A **lower value** indicates simpler, more maintainable code, while a **higher value** suggests increased risk of errors and higher testing effort.

What It Measures

- Number of **decision points** (if, while, for, switch, etc.)
- Number of **independent paths** through a program
- **Testing effort** required (minimum number of test cases)

Key Formula

For a control flow graph:

$$M = E - N + 2P$$

Where:

- **E** = Number of edges
- **N** = Number of nodes
- **P** = Number of connected components

For a **single function/module** ($P = 1$):

$$M = E - N + 2$$

Simple Interpretation

- No decision statements → **Complexity = 1**
- One `if-else` → **Complexity = 2**
- Each additional decision → **Complexity increases by 1**

Uses

- Determines **minimum number of test cases**
 - Ensures **all independent paths are tested**
 - Improves **code coverage**
 - Helps assess **risk and maintainability**
 - Guides **unit and integration testing**
-

Advantages

- Easy to compute and apply
 - Faster than Halstead metrics
 - Effective for identifying **high-risk modules**
 - Useful for **testing focus and effort estimation**
-

Limitations

- Measures **control complexity only**, not data complexity
- Can be misleading for simple decision-heavy code
- Does not fully capture **readability or design quality**

What does “*Testing effort required (minimum number of test cases)*” mean?

Cyclomatic Complexity **does NOT** mean *total* test cases.

It means:

| The minimum number of test cases needed to execute every independent path at least once.

Key idea: *Independent paths*

An **independent path** is a path through the code that introduces **at least one new decision outcome**.

Each independent path **must be tested separately**.

👉 **Cyclomatic Complexity = Number of independent paths**

👉 **Minimum test cases = Cyclomatic Complexity**

Simple examples 🚀

1 No decision

```
a = b + c;  
print(a);
```

- Paths: 1
 - Cyclomatic Complexity = 1
 - **Minimum test cases = 1**
-

2 One `if-else`

```
if (x > 0)  
    y = 1;  
else  
    y = -1;
```

- Paths:
 - `x > 0` → true
 - `x > 0` → false
- Cyclomatic Complexity = 2
- **Minimum test cases = 2**

✓ Test case 1: `x = 5`

✓ Test case 2: `x = -3`

3 Two decisions

```
if (a > b)
    x = 1;

if (c > d)
    y = 2;
```

- Independent paths = 3
- Cyclomatic Complexity = 3
- **Minimum test cases = 3**

Why? Because combinations overlap, but **3 paths are enough** to cover all decisions.

▼ Cocomo model

COCOMO Model I – Theory (Exam Oriented)

Introduction

The **Constructive Cost Model (COCOMO)** is a **parametric software cost estimation model** introduced by **Barry W. Boehm** in 1981.

COCOMO Model I is based on the empirical analysis of **63 real-world software projects** and establishes mathematical relationships between **software size** and **development effort**.

The model assumes that:

- Software is developed using **traditional procedural languages**
- The development process follows a **waterfall life cycle**
- Software size can be estimated reliably in **KLOC**

Basic Concept

COCOMO Model I assumes that:

| Effort required to build software increases non-linearly with program size

This non-linearity is captured using an exponential equation, where:

- Larger projects introduce communication overhead
 - Coordination and integration become harder
 - Management complexity increases
-

Project Classification (Modes)

COCOMO Model I classifies projects into **three modes**, based on complexity, constraints, and team experience:

1. Organic Mode

- Small and relatively simple systems
- Stable requirements
- Experienced development team
- Minimal hardware and software constraints

Examples: Payroll system, inventory system

2. Semi-Detached Mode

- Medium-sized systems
- Mixed team experience
- Moderate complexity and constraints

Examples: Compilers, database management systems

3. Embedded Mode

- Large, complex systems

- Tight hardware, software, and operational constraints
- Real-time or safety-critical systems

Examples: Aircraft control systems, medical devices

Estimation Levels in COCOMO Model I

1. Basic COCOMO (Theory)

Basic COCOMO estimates effort using only:

- Program size (KLOC)
- Project mode

It ignores environmental and human factors, making it suitable for **early feasibility studies**.

2. Intermediate COCOMO (Theory)

Intermediate COCOMO refines the estimation by incorporating **effort multipliers**, which represent cost drivers related to:

- Product reliability and complexity
- Hardware constraints
- Personnel capability and experience
- Project tools and schedule constraints

Each cost driver is rated on a scale from **very low to extra high**, and the combined effect is modeled as a multiplicative factor.

3. Detailed COCOMO (Theory)

Detailed COCOMO further improves accuracy by:

- Allocating effort across **software life cycle phases**
- Applying effort multipliers separately to each phase

This helps project managers in **resource planning and scheduling**.

Interpretation of the Effort Equation

$$\text{Effort} = a \times (\text{KLOC})^b \quad \text{Effort} = a \times (\text{KLOC})^b$$

$$\text{Effort} = a \times (\text{KLOC})^b$$

- The constant **a** represents baseline productivity
 - The exponent **b (>1)** models the **diseconomy of scale**
 - As KLOC increases, effort grows faster than linearly
-

Strengths of COCOMO Model I

- Based on real project data
 - Simple mathematical structure
 - Useful for macro-level planning
 - Easy to apply and understand
-

Limitations (Theoretical Perspective)

- Heavy dependence on accurate KLOC estimation
 - Does not support object-oriented or component-based development
 - No explicit modeling of reuse or iterative development
 - Scale behavior is fixed and not adjustable
-

Why COCOMO Model I Became Inadequate

With the evolution of software engineering:

- Languages became higher-level
- Reuse became common
- Agile and iterative models replaced waterfall

These limitations led to the development of **COCOMO II**.

▼ Fundamental Software Design Concepts

Over the evolution of software engineering, a **set of fundamental software design concepts** has emerged. These concepts have **stood the test of time** and form the **foundation of all modern design methods**, including structured and object-oriented design.

They help a software designer answer three critical questions:

1. **How should software be partitioned into components?**
2. **How should functional and data details be separated from conceptual design?**
3. **What criteria define a high-quality software design?**

As M. A. Jackson stated, *"Getting a program to work is different from getting it right."*

These design concepts provide the framework for **getting it right**.

1. Abstraction

Abstraction is a fundamental mechanism for **managing complexity** by **focusing on essential features and ignoring unnecessary details**.

Levels of Abstraction

- **High-level abstraction**
Describes the system using problem-domain language
- **Low-level abstraction**
Combines problem-domain terms with implementation details
- **Lowest-level abstraction**
Directly implementable design

Types of Abstraction

- **Procedural abstraction**
A named sequence of instructions with a specific purpose

Example: `open()` hides detailed steps required to open a door

- **Data abstraction**

A named collection of data that describes an object

Example: `door` with attributes like type, size, mechanism

Importance

- Reduces complexity
 - Improves clarity
 - Encourages reuse
 - Forms the basis for modularity and information hiding
-

2. Software Architecture

Software architecture refers to the **overall structure of the system**, the **organization of its components**, and **how they interact**.

Architecture Defines:

- Major system components
- Component interactions
- Data organization
- Control flow

Architectural Properties (Shaw & Garlan)

1. **Structural properties**

Components, modules, objects, and their connections

2. **Extra-functional properties**

Performance, reliability, security, scalability, adaptability

3. **Families of systems**

Reusable architectural patterns for similar applications

Architectural Models

- Structural models
- Framework models
- Dynamic models
- Process models
- Functional models

Key Point:

Architecture provides the **highest return on investment** with respect to quality, cost, and schedule.

3. Design Patterns

A **design pattern** is a **named, reusable solution** to a recurring design problem within a specific context.

Characteristics

- Captures design knowledge
- Proven through experience
- Provides guidance, not code

Benefits

- Saves design time
 - Improves design quality
 - Encourages reuse
 - Helps designers communicate effectively
-

4. Separation of Concerns

Separation of concerns states that **complex problems are easier to solve when divided into smaller, independent parts.**

Key Ideas

- Each concern represents a requirement or behavior
- Smaller problems require less effort
- Combined problems are more complex than individual ones

Caution

- Over-separation leads to difficult integration

Importance

- Forms the basis for:
 - Modularity
 - Functional independence
 - Refinement
 - Aspect-oriented design
-

5. Modularity

Modularity is the **most common application of separation of concerns**.

Definition

Software is divided into **independent, named modules** that are later integrated.

Advantages

- Easier understanding
- Simplified testing and debugging
- Better maintainability
- Easier change management

Modularity Trade-off

- **Too few modules** → high complexity

- **Too many modules** → high integration cost

There exists an **optimal number of modules (M)** that minimizes total development cost.

6. Information Hiding

Information hiding suggests that **modules should hide internal details** (data structures and algorithms) from other modules.

Key Principles

- Only necessary information is exposed
- Internal logic remains inaccessible
- Modules communicate through well-defined interfaces

Benefits

- Reduces ripple effect
 - Limits error propagation
 - Simplifies maintenance
 - Improves reliability
-

7. Functional Independence

Functional independence means that **each module performs a single, well-defined task** and interacts minimally with other modules.

Achieved Through:

- **High cohesion**
A module focuses on one function
- **Low coupling**
Minimal dependency between modules

Why Important?

- Easier development
- Easier testing
- Easier maintenance
- Higher reusability

Golden Rule:

| High cohesion + Low coupling = Good design

8. Refinement

Refinement is a **top-down design approach** where a system is developed by **progressively adding detail**.

Process

- Start with a high-level description
- Gradually elaborate into detailed design
- End with implementable logic

Relationship with Abstraction

- Abstraction hides details
 - Refinement reveals details
-

9. Aspects

Some requirements **cut across multiple modules** and cannot be isolated easily.

These are called **crosscutting concerns**.

Examples

- Security
- Authentication

- Logging
- Error handling

An **aspect** is a representation of such a concern.

Aspect-Oriented Design

- Keeps crosscutting logic separate
 - Avoids scattered and tangled code
 - Improves modularity
-

10. Refactoring

Refactoring is the process of **improving internal design without changing external behavior**.

Objectives

- Remove redundancy
- Improve cohesion
- Simplify structure
- Enhance maintainability

Common Refactoring Actions

- Splitting large modules
- Eliminating unused logic
- Simplifying complex structures

Usage

- Widely used in Agile development
- Improves long-term design quality

▼ What is Agility ?

Agility in software engineering is the **ability of a development team to respond effectively and efficiently to change** throughout the software development life cycle.

Agility promotes **rapid and incremental delivery of working software**, allowing feedback to be incorporated early and frequently. It also supports flexible planning, understanding that detailed long-term plans are difficult to maintain in an uncertain environment.

However, agility does not mean a lack of discipline or structure. It requires a **well-defined process, continuous testing, and disciplined development practices** to ensure quality while adapting to change.

▼ Agility and the Cost of Change

In traditional software development, it is widely accepted that the **cost of making changes increases nonlinearly as the project progresses**. This means that changes made **early in the development cycle**—such as during the requirements phase—are relatively easy and inexpensive. At this stage, changes may only involve modifying requirement documents, updating use cases, or extending a list of functions, which requires minimal effort, time, and cost.

However, when changes are introduced **later in the project**, such as during validation or testing, the situation becomes much more complex. A single change request may require **modifications to the software architecture**, redesigning and rebuilding multiple components, updating interfaces, and creating new test cases. Additionally, there is a higher risk of introducing unintended side effects, which further increases the cost and time required. As a result, the cost of change rises sharply in conventional development processes.

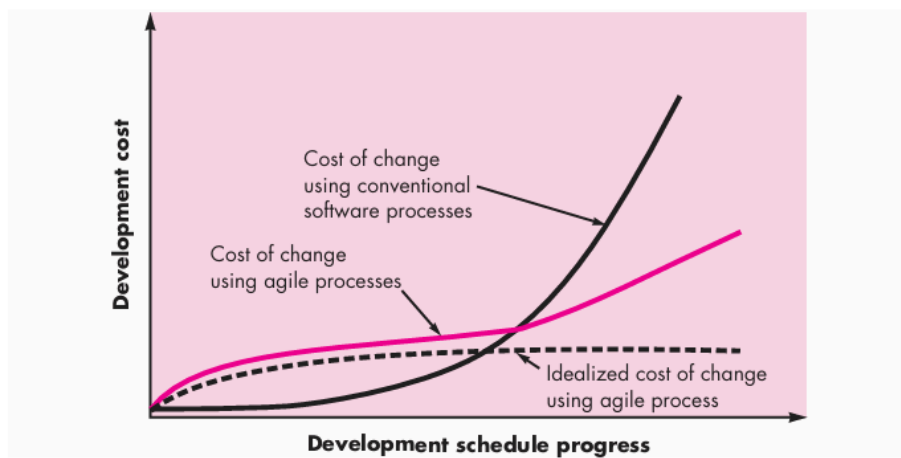
Agile proponents argue that a **well-designed agile process significantly reduces this cost escalation** by “flattening” the cost-of-change curve. Agile achieves this through **incremental delivery**, where software is developed and released in small, manageable increments. Because changes are handled within a limited scope of an increment, they are easier to control and less expensive to implement.

Furthermore, agile practices such as **continuous unit testing, frequent integration, and pair programming** help detect defects early and maintain code quality. These practices reduce the ripple effect of changes and prevent costly rework at later stages.

FIGURE 3.1

Change costs as a function of time in development

note:
“Agility is dynamic, content specific, aggressively change embracing, and growth oriented.”
—Steven Goldman et al.



not adversely affect the outcome of the project. But what if we fast-forward a number of months? The team is in the middle of validation testing (something that occurs relatively late in the project), and an important stakeholder is requesting a major functional change. The change requires a modification to the architectural design of

▼ What is an agile process ?

An **agile process** is a software development process that is **adaptive, incremental, and people-centric**, designed to effectively handle the uncertainty and frequent changes that occur in most software projects.

Key Assumptions Behind Agile Software Processes

Agile software processes are based on certain realities observed in most software projects:

1. Uncertainty of requirements

It is very difficult to predict which requirements will remain stable and which will change. Customer priorities also evolve as the project progresses due to market changes, feedback, or new understanding of needs.

2. Interleaving of design and construction

For many software systems, design and coding cannot be completely separated. Design decisions are often validated only when construction begins.

3. Unpredictability of development activities

Activities such as analysis, design, coding, and testing are not fully predictable from a planning perspective.

Because of these assumptions, **traditional rigid processes fail**, and a process that can **adapt to change** is required.

Need for Adaptability and Incremental Development

To manage unpredictability, an agile process must be **adaptable**.

This is achieved through:

- **Incremental development**
- **Short delivery cycles**
- **Operational prototypes or working software increments**

Each increment allows the customer to:

- Evaluate the software
- Provide feedback
- Influence future adaptations of both the product and the process

This iterative feedback loop ensures that adaptation keeps pace with change.

Agility Principles (Agile Alliance)

The Agile Alliance defines **12 core principles** that represent the agile mindset:

1. Customer satisfaction through early and continuous delivery
2. Welcoming changing requirements, even late in development
3. Frequent delivery of working software
4. Daily collaboration between business people and developers
5. Trust and support motivated individuals
6. Face-to-face communication as the most effective method
7. Working software as the primary measure of progress
8. Sustainable development with a constant pace
9. Continuous attention to technical excellence and good design
10. Simplicity—doing only what is necessary
11. Self-organizing teams produce the best designs and architectures
12. Regular reflection and process improvement

Not all agile models apply all principles equally, but together they define the **agile spirit**.

Human Factors in Agile Development

Agile development strongly emphasizes **people over processes**. The process is shaped around the team, not the other way around. For agility to succeed, certain traits must exist within the team:

1. **Competence**

Team members must possess technical skills, domain knowledge, and understanding of the chosen process.

2. **Common focus**

All team members should share a single goal: delivering a working software increment on time.

3. **Collaboration**

Continuous communication and cooperation among team members and stakeholders are essential.

4. **Decision-making ability**

Agile teams must have autonomy to make technical and project-related decisions.

5. **Fuzzy problem-solving ability**

Teams must handle ambiguity and accept that problems and solutions may change over time.

6. **Mutual trust and respect**

A successful agile team is a “jelled” team, where trust and respect make the team stronger as a whole.

7. **Self-organization**

Agile teams:

- Organize their own work
- Define their own processes
- Commit to achievable goals

Self-organization improves motivation, collaboration, and morale, and ensures that teams take responsibility for their commitments.

▼ What is Requirements Engineering?

Requirements Engineering (RE) is the systematic process of **discovering, analyzing, documenting, validating, and managing software requirements.**

Tasks in Requirements Engineering:

Requirements engineering consists of **seven interrelated tasks**:

1. Inception

Inception is the starting point of a software project. It begins when:

- A business need is identified, or
- A new market or service opportunity is discovered

The goal of inception is to:

- Understand the problem
 - Identify stakeholders
 - Establish initial communication
 - Gain a basic idea of the desired solution
-

2. Elicitation

Elicitation involves **gathering requirements from stakeholders** such as customers, users, and managers. Although it appears simple, it is actually difficult because stakeholders may:

- Have unclear needs
- Express requirements inconsistently
- Focus on solutions instead of problems

A key part of elicitation is understanding **business goals**, which may be:

- Functional (what the system should do)
- Nonfunctional (performance, security, usability, reliability)

Goals help:

- Explain requirements clearly
- Resolve conflicts

- Guide elaboration, validation, and negotiation

Agility plays an important role here because **new requirements often emerge during iterative development**.

3. Elaboration

Elaboration focuses on creating a **refined requirements model** that describes:

- Software functions
- Behavior
- Information flow

This is done by:

- Developing user scenarios
- Identifying analysis classes (business entities)
- Defining attributes and services
- Establishing relationships between classes

The objective is to **understand the problem clearly**, not to over-design. Excessive detail should be avoided.

4. Negotiation

Stakeholders often request:

- More features than resources allow
- Conflicting requirements

Negotiation helps resolve these conflicts by:

- Prioritizing requirements
- Evaluating cost and risk
- Modifying or combining requirements

The goal is a **mutually acceptable solution**, where no party fully loses and all achieve reasonable satisfaction.

5. Specification

Specification is the **formal documentation of requirements**. It may take many forms:

- Written documents
- Graphical models
- Usage scenarios
- Prototypes
- Formal mathematical models

The format depends on:

- Project size
- System complexity
- Development environment

Large systems usually require a detailed Software Requirements Specification (SRS), while smaller systems may only need usage scenarios.

6. Validation

Validation ensures that requirements are:

- Correct
- Complete
- Consistent
- Unambiguous
- Realistic and testable

The primary validation technique is a **technical review**, involving developers, customers, users, and stakeholders.

Validation identifies:

- Missing requirements
- Conflicts and inconsistencies

- Vague or unrealistic statements

For example:

- "The software should be user friendly" is too vague
- "Intrusion probability < 0.0001" may be unrealistic or unnecessary

Such requirements must be clarified, quantified, or replaced with more practical alternatives.

7. Requirement management(also)

▼ Requirement management

Requirements Validation

Identifying Stakeholders

A **stakeholder** is anyone who benefits directly or indirectly from the system. Stakeholders include managers, customers, end users, developers, testers, support engineers, and consultants. Each stakeholder has:

- A different perspective
- Different priorities
- Different risks if the project fails

At project inception, an initial list of stakeholders is created and expanded as new stakeholders are identified.

Recognizing Multiple Viewpoints

Since stakeholders have different goals, requirements are gathered from **multiple viewpoints**, such as:

- Marketing (marketable features)
- Business managers (cost and schedule)
- End users (usability and familiarity)

- Developers (technical feasibility)
- Support engineers (maintainability)

These viewpoints may result in **conflicting or inconsistent requirements**, which must be categorized and resolved to produce a consistent requirement set.

Working Toward Collaboration

Effective requirements engineering requires **collaboration among stakeholders and developers**. The role of the requirements engineer is to:

- Identify common requirements
- Detect conflicts and inconsistencies
- Facilitate resolution through discussion or prioritization

In some cases, a **project champion** makes final decisions. Techniques such as **Planning Poker** are used to prioritize requirements by assigning points based on stakeholder importance.

▼ Data Flow diagram

Data Flow Diagram (DFD)

Definition

A **Data Flow Diagram (DFD)** is a **graphical representation** of how **data flows through a system**. It shows **how data enters the system, how it is processed, where it is stored, and how it leaves the system**, without showing implementation or program logic.

DFD focuses on **WHAT happens to data**, not **HOW it is implemented**.

Purpose of DFD

- To understand **system functionality**
- To analyze **data movement** in a system

- To identify **inputs, outputs, processes, and data storage**
 - To communicate system requirements clearly between **analysts, developers, and users**
 - To help in **system design and documentation**
-

Main Components of DFD

1 External Entity

- Represents a **person, organization, or system** outside the boundary of the system
- Sends data to or receives data from the system
- Does **not** perform processing

 Examples:

- Customer
- Bank User
- Payment Gateway
- Admin

 Drawn as: **Rectangle**

2 Process

- Represents an **operation or activity** that transforms input data into output data
- Each process must have:
 - At least **one input**
 - At least **one output**

 Examples:

- Verify Login
- Process Payment

- Transfer Funds
- Generate Statement

 Drawn as: **Circle or rounded rectangle**

3 Data Flow

- Represents the **movement of data** between entities, processes, and data stores
- Must be **named**
- Direction is shown using arrows

 Examples:

- Login Details
- Payment Request
- Transaction Status
- Account Balance

 Drawn as: **Arrow**

4 Data Store

- Represents a **place where data is stored**
- Used by processes to read or write data

 Examples:

- Account Database
- Transaction Records
- User Credentials

 Drawn as: **Open-ended rectangle / two parallel lines**

Levels of Data Flow Diagram

1 Context Diagram (Level -1)

- Represents the **entire system as a single process**
- Shows:
 - External entities
 - Data flows
- Does **not show** internal processes or data stores

 Purpose:

- Defines **system boundary**
 - Gives **high-level overview**
-

2 Level-0 DFD

- Expands the context diagram
- Breaks the system into **major sub-processes**
- Shows:
 - Multiple processes
 - Data stores
 - External entities

 Purpose:

- Shows **main system functions**
-

3 Level-1 DFD

- Further decomposes a **single process** from Level-0
- Shows detailed sub-processes

 Example:

Process **2.0 Transfer Funds** →

- 2.1 Validate Account

- 2.2 Check Balance
 - 2.3 Transfer Amount
 - 2.4 Update Records
-

4 Level-2 and beyond

- Used when processes are **very complex**
 - Each level increases detail
-

Rules of DFD (Very Important for Exams ⚠)

1. A process must have **input and output**
 2. Data cannot flow:
 - From **entity to entity**
 - From **store to store**
 - From **entity directly to data store**
 3. Data flows must be **named**
 4. DFDs must be **balanced**
 - Inputs and outputs at one level must match the next level
 5. DFD does **not show control flow, loops, or decisions**
-

Advantages of DFD

- Easy to understand
 - Improves communication
 - Helps in requirement analysis
 - Independent of programming language
 - Identifies missing or redundant processes
-

Limitations of DFD

- Does not show timing or sequence
- Not suitable for real-time systems
- Becomes complex for very large systems
- Does not represent decision logic clearly

▼ Use Case diagram

Use Case Diagram

Definition

A **Use Case Diagram** is a **UML (Unified Modeling Language) diagram** that shows **how users (actors) interact with a system**. It describes **what the system does from the user's point of view**, not how it is internally implemented.

In simple words:

👉 It shows **who can use the system and what they can do**.

Purpose of Use Case Diagram

- To understand **system functionality**
 - To identify **user requirements**
 - To define **system scope and boundaries**
 - To improve communication between **users, analysts, and developers**
 - To act as a base for **requirement analysis and design**
-

Main Components of a Use Case Diagram

1 Actor

- An **actor** is any **user or external system** that interacts with the system

- Actors are **outside the system boundary**
- An actor can be:
 - Human (Customer, Admin)
 - Another system (Payment Gateway)

 Examples:

- Account Holder
- Bank Admin
- Payment Gateway

 Drawn as: **Stick figure**

2 Use Case

- A **use case** represents a **function or service** provided by the system
- It describes **what the system does**, not how

 Examples:

- Login
- Check Balance
- Transfer Funds
- Pay Bills
- Apply for Loan

 Drawn as: **Oval**

3 System Boundary

- Defines the **scope of the system**
- Everything **inside** the box belongs to the system
- Actors stay **outside** the box

 Drawn as: **Rectangle**

4 Association

- Shows **interaction** between actor and use case
- Indicates that the actor **participates in** that use case

 Drawn as: **Simple line**

Relationships in Use Case Diagram

1 Include (<<include>>)

- One use case **always uses another**
- Used to **avoid repetition**

 Example:

- "Transfer Funds" **includes** "Authenticate User"

Meaning:

Authentication is **mandatory** for fund transfer.

2 Extend (<<extend>>)

- One use case **optionally extends** another
- Happens only under **specific conditions**

 Example:

- "Apply for Loan" **extends** "Check Eligibility"

Meaning:

Eligibility check happens **only when loan is applied**.

3 Generalization

- Shows **inheritance** between actors or use cases
- Child actor/use case inherits behavior from parent

 Example:

- User → Admin
-

Example: Online Banking System (Simple)

Actors

- Account Holder
- Payment Gateway

Use Cases

- Login
 - Change Password
 - Check Balance
 - Transfer Funds
 - Pay Bills
 - Request Statement
 - Request Cheque Book
-

Rules of Use Case Diagram

1. Use cases must be **clear and action-oriented**
 2. Actors must be **outside system boundary**
 3. Avoid technical or implementation details
 4. Use case diagram shows **behavior**, not data flow
 5. One actor can interact with **multiple use cases**
-

Advantages of Use Case Diagram

- Easy to understand
- User-friendly

- Helps in requirement gathering
 - Clearly defines system functionality
 - Useful for planning and testing
-

Limitations of Use Case Diagram

- Does not show internal logic
- No data flow representation
- Cannot show sequence of actions
- Not suitable for detailed design

DOCUMENTATION :

1. UML Use Case Diagram (top part)

This part shows:

- **Actor:** User
- **Use cases:**
 - Register
 - Login
 - Search Train
 - Book Train

👉 This answers:

“What can the user do?”

But it does NOT explain:

- steps
- conditions
- errors

- priority

That's why the next parts exist.

2. Preliminary Use Case

This is just a **rough list of main actions** before writing details.

Example (in your notes, left blank):

1. Register
2. Login
3. Book Ticket

👉 Purpose:

- Identify important use cases
 - Decide which ones need full documentation
-

3. Formal Use Case / Documentation (MOST IMPORTANT)

This is the **detailed explanation of ONE use case**.

In your image, the documented use case is:

Use case: Book Train Ticket

Now each line explains something specific 👉

a) Iteration: 2nd iteration

- Means: this use case is implemented/refined in the **second development cycle**
 - Used in agile projects
-

b) Primary actor: Customer

- The **main person** who starts the use case
-

c) Goal in context: To book a ticket

- The **purpose** of the use case
-

d) Precondition: Registration availability (log in to system)

- What **must be true before** the use case starts
 - Example:
 - User must be registered
 - User must be logged in
-

e) Trigger: To travel

- The **event that starts** the use case
 - Example: user wants to travel
-

4. Scenario (Normal Flow)

This is the **step-by-step success path**.

From your image:

1. Name, age
2. Window preference
3. Payment
4. Acknowledgement

👉 This answers:

"How does booking normally happen?"

5. Exceptions (Alternative / Error Flows)

Things that **can go wrong**:

1. Network bandwidth is low
2. Payment processing fails

3. Server overloading

👉 These are **not shown in the diagram**, so they are written here.

6. Priority: High

- Indicates how important this use case is
 - Helps project planning
-

7. When available: After 2nd iteration

- Tells **when this feature will be ready**
-

8. Frequency of use: High

- How often users use this feature
 - Booking tickets happens frequently
-

9. Channel to actor: Internet

- How the user interacts with the system
-

10. Secondary actor: Admin

- Someone who supports the system indirectly
-

11. Channel to secondary actor: Internet

- How admin connects
-

12. Open Issues: Time taken

- Known problems not yet solved
- Helps future improvement

▼ Activity diagram

1 What is an Activity Diagram? (Simple meaning)

An **Activity Diagram** shows:

- **How work flows**
- From **start to end**
- Step by step
- Including **decisions and parallel actions**

👉 It is like a **flowchart**, but used in **UML**.

2 What does it model?

An activity diagram models:

- **Behavior of a system**
- Or **workflow of a process**
- Or **logic inside a use case**

So it answers:

| "In what order do actions happen?"

3 Where does it belong?

- It is part of **Behavioral Modeling**
- More specifically: **Control-flow behavior**

📌 Unlike use cases (user view), activity diagrams show **internal flow**.

4 Main components of an Activity Diagram

- ♦ **1. Initial Node**
 - Solid black circle

- Shows where the activity starts
-

♦ 2. Activity / Action

- Rounded rectangle
- Represents a step or operation

Example:

- Enter login details
 - Verify password
 - Process payment
-

♦ 3. Control Flow

- Arrow showing the sequence of actions
-

♦ 4. Decision Node

- Diamond shape
- Used for conditions (if / else)

Example:

- Payment successful?
 - Yes → Confirm order
 - No → Show error
-

♦ 5. Merge Node

- Diamond
 - Combines alternative paths back into one
-

♦ 6. Fork Node

- Thick horizontal or vertical bar

- Splits flow into **parallel activities**

Example:

- Send confirmation email
 - Update database
- (both happen at the same time)
-

◆ 7. Join Node

- Thick bar
 - Joins parallel flows back into one
-

◆ 8. Final Node

- Circle with a dot inside
 - Marks end of activity
-

5 Simple example: Online Banking – Pay Bill

Flow:

1. Start
2. User selects "Pay Bill"
3. Enter bill details
4. Verify balance
5. Decision: Sufficient balance?
 - Yes → Process payment
 - No → Show error
6. Update account
7. End

👉 This whole flow is shown clearly using an activity diagram.

▼ Swimlane diagram

What is a Swimlane Diagram?

A **Swimlane diagram** is a **type of Activity Diagram (UML)** that shows **who does what** in a process.

- 👉 It **divides the activity diagram into lanes**, and
- 👉 **each lane represents a responsible entity** such as:

- a person
- a department
- a system
- or a role

So instead of only showing *what* happens, it also shows **who is responsible** for each step.

Why is it called "Swimlane"?

Think of a swimming pool:

- Each swimmer stays in **their own lane**
- Actions don't mix across lanes

Similarly:

- Each **lane = one actor**
- Activities are placed **inside the lane of the actor who performs them**

▼ State diagram

1. Identify States

From the paragraph, a **state** is a condition or situation of an object at a specific time. There are two types:

- **Passive State:** Attributes of the object (e.g., Player's position, magic points)

- **Active State:** Behavior or activity of the object (e.g., moving, at rest, injured)

Example from paragraph (ControlPanel class):

- Active states: `Reading`, `Comparing`, `Locked`, `Selecting`

Tip: For a state diagram, we focus on **active states**.

2. Identify Events (Triggers)

An **event or trigger** causes the object to change from one state to another.

- Example: `Key hit`, `Password entered`, `Timer ≤ lockedTime`

Tip: Events are written on the arrow (transition line) connecting states.

3. Identify Guards

A **guard** is a Boolean condition that must be true for the transition to occur.

- Example:

```
Password = incorrect & numberOfTries < maxTries
```

- Guards are optional but important for conditional transitions.

Symbol in UML:

- Written in square brackets on the transition arrow:

```
[Password = incorrect & numberOfTries < maxTries]
```

4. Identify Actions

An **action** is an operation performed during or as a result of the transition.

- Example: `Do: validatePassword()`
- In UML, actions are usually written after a slash `/` on the arrow:

```
/validatePassword()
```

5. Draw the Diagram

UML Symbols

1. **States:** Rounded rectangles (ellipses or rectangles with rounded corners)

```
+-----+  
|      Reading      | +-----+
```

2. **Initial State:** Solid black circle
3. **Final State:** Black circle with an outer ring
4. **Transitions:** Arrows from one state to another
5. **Event [Guard] / Action:** Written on the arrow:

```
Event[Guard] / Action
```

6. Steps to Draw

1. Draw **initial state** (solid black circle)
2. Draw **active states** as rounded rectangles
 - Example: `Reading`, `Comparing`, `Locked`, `Selecting`
3. Connect states with **arrows** representing **transitions**
4. Label **arrows** with:
 - **Event (trigger)**
 - **[Guard condition]** (if any)
 - **/Action** (if any)

7. Example from Paragraph (simplified)

```
(initial)
  ●|
  v+-----+|  Reading|+-----+|
Key hit|
  v+-----+| Comparing|+-----+| Password correct/ validatePassword()
  v+-----+| Selecting|+-----+|
  v+-----+| Locked|+-----+|
  v
(final)
```

With guards and actions explicitly:

```
Reading
|
| Key hit [Password input = 4 digits] / validatePassword()
vComparing
|
| Password = correct / activate()
vSelecting
|
| NumberOfTries > maxTries
vLocked
```

✓ Summary of Symbols

Symbol	Meaning
Rounded rectangle	Active state
Solid black circle	Initial state
Circle with ring	Final state
Arrow	Transition (state change)
Event [Guard] / Action	Label on arrow showing trigger, condition, and action

1. The Three Compartments

A UML state rectangle can have **three horizontal compartments**:

```

+-----+
|      StateName      | <-- **1. Name of the state**
+-----+
|Internal Activities | <-- **2. Entry/Do/Exit actions**
+-----+
|Internal Events    | <-- **3. Internal transitions**
+-----+

```

Compartment 1: State Name

- This is **mandatory**.
- Simply the **name of the state**, describing what the object is “doing” or “experiencing”.
- Example: `Reading` , `Locked` , `Moving`

Compartment 2: Internal Activities (Optional)

This compartment can hold **three types of internal behaviors**:

1. **Entry Action (`entry:`)** – what happens when the object enters the state.
Example: `entry: startTimer()`
2. **Do Action (`do:`)** – what the object continuously does while in this state.
Example: `do: monitorKeyPress()`
3. **Exit Action (`exit:`)** – what happens when the object leaves the state.
Example: `exit: stopTimer()`

Think of this compartment as **state-specific behaviors** that are not dependent on a transition.

Compartment 3: Internal Transitions / Events (Optional)

- Shows **events that are handled within the state** without leaving it.
- Can include **self-transitions** or **responses to events** that trigger internal actions.
- Example:

```
keyHit / validateInput()timeout / resetTimer()
```

This compartment is optional and is used for **more complex states** where internal events or actions need to be explicitly shown.

2. Why Divide the Rectangle?

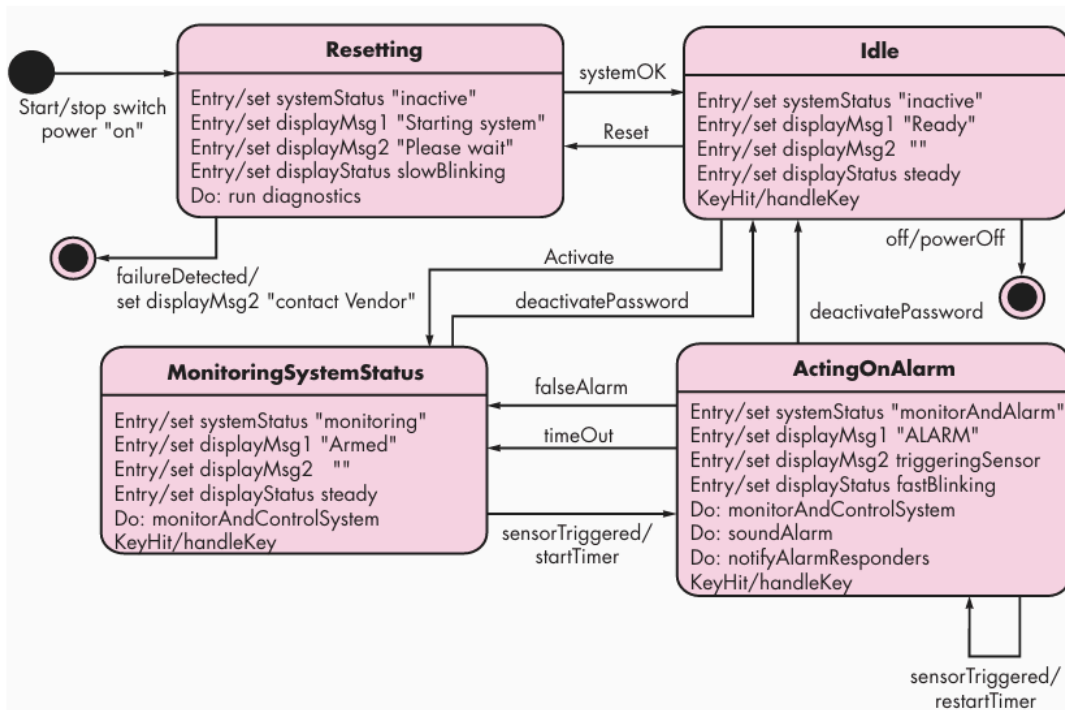
- **Clarity:** You separate **state identity**, **behaviors while in state**, and **responses to events**.
- **Documentation:** Makes it easier to show **entry, do, exit actions** without cluttering the diagram.
- **Behavior modeling:** Helps model **active states with ongoing processes**, not just transitions.

3. Quick Example Using ControlPanel

```
+-----+
|   Reading   | <-- State Name
+-----+
| entry: displayPrompt()do: monitorKeyPress() |
| keyHit / readDigit() |
+-----+
```

- `Reading` is the **state name**
- `entry:` and `do:` are **internal actions**
- `keyHit / readDigit()` is an **internal event/transition**

FIGURE 7.4 State diagram for SafeHome security function



▼ Design Concepts

Fundamental Software Design Concepts

Software design is the process of **transforming requirements into a blueprint** for constructing the software system. To manage complexity and ensure quality, a number of **fundamental design concepts** are applied.

1. Abstraction

Abstraction is the process of **focusing on essential features** of a system while **suppressing unnecessary details**. It allows designers to think at different levels of detail during design.

- **Data abstraction:** Defines data objects using attributes and operations without specifying internal representation.
- **Procedural abstraction:** Defines a sequence of operations by name without revealing implementation details.

- Abstraction helps in **reducing complexity**, improving understandability, and enabling step-wise refinement.
-

2. Architecture

Software architecture describes the **overall structure of the system** and the relationships among its components.

- Defines components, connectors, and data flow.
- Provides a framework for:
 - Detailed design
 - Development
 - Maintenance
- Captures:
 - **Structural properties** (components and connections)
 - **Behavioral properties** (runtime interaction)
 - **Nonfunctional properties** (performance, security, reliability)

3. Separation of Concerns

- Acts as the foundation for design decisions and system evolution.
-

Separation of concerns is the principle of **dividing a complex problem into distinct parts**, each addressing a specific concern.

- Each concern can be designed and solved independently.
- Reduces complexity and improves clarity.
- Forms the basis for:
 - Modularity
 - Information hiding
 - Aspect-oriented design

- Excessive separation can increase integration complexity and must be balanced.

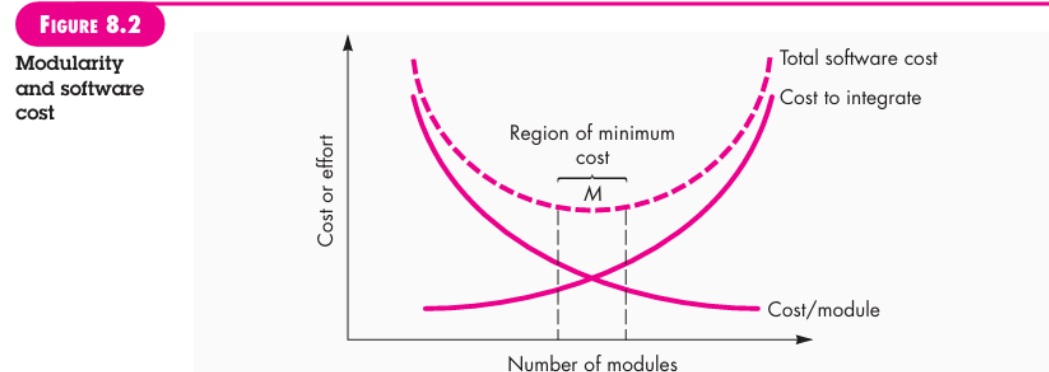
4. Modularity

Modularity involves dividing the system into **discrete modules**, each responsible for a specific task.

- A module is a **logically independent part** of a program.

However, modularization cannot be increased indefinitely. Although the **cost to develop individual modules decreases** as the number of modules increases, the **cost of integrating these modules increases**. As a result, the **total development cost follows a curve**, where there exists an optimal number of modules, denoted as **M**, at which the total cost is minimum.

Although it is not possible to precisely calculate the ideal value of **M**, designers should aim to modularize software **near the region of minimum cost**, avoiding both undermodularity and overmodularity.



5. Information Hiding

Information hiding requires that **each module hides its internal data structures and processing logic** from other modules.

- Only necessary information is exposed through well-defined interfaces.

- Internal details can be changed without affecting other modules.
- Supports:
 - Easier maintenance
 - Reduced error propagation
 - Improved robustness

Role of Coupling and Cohesion in Information Hiding

- **Low coupling** ensures modules depend minimally on each other.
- **High cohesion** ensures each module performs a single, well-defined function.
- Together, low coupling and high cohesion **enable effective information hiding** by limiting inter-module dependency and maximizing internal consistency.

8. Functional Independence

A software system is said to have functional independence when its **modules perform a single, well-defined function** and have **minimal interaction with other modules**.

Functional independence is achieved when a module has:

- **High cohesion**
- **Low coupling**

Benefits:

- Independent testing
- Independent maintenance
- Reduced ripple effects
- Better reliability

Functional independence is a **direct goal of modular design**.

9. Refinement

Refinement is a **top-down design technique** where the system is developed by gradually adding detail.

- Begins with high-level functions.
 - Moves toward implementable details.
 - Complements abstraction:
 - Abstraction hides detail.
 - Refinement reveals detail gradually.
-

10. Design Patterns

Design patterns are **standard, reusable solutions** to commonly occurring design problems.

- Provide proven approaches rather than reinventing solutions.
- Improve consistency and quality.
- Examples: Singleton, Observer, Factory.
- Patterns exist within a defined **context and constraints**.

11. Aspects:

In software design, an **aspect** is used to handle **crosscutting concerns**—that is, concerns that **affect multiple modules across the entire system** rather than belonging to a single module.

A **crosscutting concern** is a requirement or behavior that **cannot be cleanly confined to one module** because it must be applied to many different parts of the system.

Examples of Crosscutting Concerns

Common examples include:

- **Logging** – recording activities across many modules.

Traditional modularization focuses on dividing software into functional modules. However, crosscutting concerns do not fit well into this structure because:

- Their code gets **scattered** across many modules.

Aspect-Oriented Design Solution

Aspect-Oriented Design (AOD) addresses this problem by **separating crosscutting concerns into independent aspects**.

An **aspect** encapsulates:

- The concern itself (e.g., security logic)
- The points in the program where it should be applied

These aspects are then **woven** into the main program at appropriate points without modifying the core business logic of individual modules

12. Refactoring

Refactoring improves the **internal structure of software** without changing its external behavior.

- Enhances:
 - Readability
 - Maintainability
 - Performance
- Common in agile development.
- Helps reduce technical debt.

13. Object-Oriented Design Concepts

Object-oriented design organizes software around **objects and classes**.

Key concepts:

- **Encapsulation** – Bundling data and methods
- **Inheritance** – Reusing and extending behavior
- **Polymorphism** – One interface, many implementations
- **Message passing** – Object interaction

▼ Design classes

In software engineering, the **requirements model** identifies a set of **analysis classes** that represent elements of the problem domain.

As development moves from

analysis to design

, these analysis classes are

refined into design classes

.

Design classes add technical detail

such as data structures, method definitions, and interactions so that the system can be

implemented and executed.

Types of Design Classes

Five major types of design classes are defined, each corresponding to a **layer in the design architecture**:

1. User Interface Classes

User interface (UI) classes define all abstractions required for **human-computer interaction (HCI)**.

They handle:

- Screens, windows, forms, dialogs
- User input and output

- Interaction metaphors (e.g., order forms, checkbooks, dashboards)

These classes focus on **presentation and interaction**, not business logic.

2. Business Domain Classes

Business domain classes are **refinements of analysis classes**.

They:

- Represent real-world business entities
- Define **attributes (data)** and **services (methods)**
- Implement core business rules and behavior

These classes form the **heart of the application logic**.

3. Process Classes

Process classes implement **lower-level business logic** that supports business domain classes.

They:

- Coordinate workflows
- Manage complex operations involving multiple business objects
- Handle control logic rather than data storage

They are especially useful when business rules are too complex to fit inside a single domain class.

4. Persistent Classes

Persistent classes represent **data stores** that survive beyond program execution.

They:

- Manage databases, files, or external storage
- Provide methods for saving, retrieving, updating, and deleting data
- Separate data persistence logic from business logic

This improves maintainability and scalability.

5. System Classes

System classes implement **software management and control functions**.

They:

- Handle communication with operating systems, networks, and external systems
- Manage memory, error handling, security, and resource allocation
- Enable the system to operate within its execution environment

These classes are typically not visible to end users.

.

Transformation from Analysis to Design Classes

As the design model evolves:

- **Abstraction level decreases**
- Business-oriented descriptions are replaced by **implementation-oriented details**
- Design classes specify exact methods, attributes, interfaces, and collaborations

Thus, design classes serve as a **blueprint for coding**.

Well-Formed Design Classes

According to Arlow and Neustadt, every design class should be **well-formed**, satisfying four key characteristics:

1. Complete and Sufficient

A design class must:

- Include **all attributes and methods** logically associated with its purpose (**complete**)

- Contain **only what is necessary**, no redundant or unrelated behavior (**sufficient**)

This ensures clarity and avoids bloated class designs.

2. Primitiveness

Each method should:

- Perform **one specific service**
- Provide **only one way** to perform that service

This avoids duplicate functionality and simplifies maintenance.

3. High Cohesion

A design class should have:

- A **small, focused set of responsibilities**
- Methods that operate on the same related data

High cohesion improves readability, reliability, and reusability.

4. Low Coupling

Design classes must:

- Collaborate only when necessary
- Have **minimal dependency** on other classes

Low coupling:

- Simplifies testing and maintenance
- Reduces ripple effects when changes occur
- Improves system flexibility

This principle is reinforced by the **Law of Demeter**, which states that a method should communicate only with **closely related ("neighbor") classes**.

▼ Design model

Software Design Model

The **software design model** is the blueprint for building software, similar to an **architect's plan for a house**.

The design model provides **multiple views of the system**, helping developers, testers, and maintainers understand how the software is structured and how it works internally.

Dimensions of the Design Model

The design model can be viewed in **two dimensions**:

1. Process Dimension

- Shows how the design **evolves over time** as design activities are performed.
- Indicates that design is **not a single step**, but a sequence of tasks such as architectural design, interface design, component design, and deployment design.

2. Abstraction Dimension

- Represents the **level of detail** in the design.
 - Design starts at a **high level of abstraction** and is refined into **implementation-level detail**.
 - The boundary between **analysis and design** is sometimes clear and sometimes gradual, as analysis models often blend into design models.
-

Design Model and UML

- The design model uses many of the **same UML diagrams** as the analysis model.
- However, in design:

- Diagrams are **more detailed**
 - Implementation-specific information is added
 - Architecture, components, interfaces, and interactions are emphasized
-

Development of Design Elements

- Design elements are **not always created sequentially**.
 - Architectural design is usually done first.
 - Interface design and component-level design often occur **in parallel**.
 - Deployment design is usually completed **after other design elements are stable**.
 - **Design patterns** can be applied at any stage of design to solve recurring problems efficiently.
-

Design Modeling Principles

The following **ten principles** guide good software design:

Principle 1: Traceability to Requirements

- Every design element must be traceable to the **requirements model**.
- Ensures that all requirements are correctly implemented.

Principle 2: Architecture First

- Architecture forms the **skeleton of the system**.
- It influences performance, maintainability, testing, and system behavior.
- Component-level design should follow architectural design.

Principle 3: Data Design Is Critical

- Data structures and data flow must be carefully designed.
- Poor data design leads to inefficient processing and complex logic.

Principle 4: Careful Interface Design

- Interfaces control how components communicate.
- Good interfaces reduce errors, simplify integration, and improve testability.

Principle 5: User Interface Focus

- UI must be designed for **ease of use**.
- A poor interface can make even a well-designed system seem unusable.

Principle 6: Functional Independence

- Components should perform **one well-defined function**.
- Achieved through **high cohesion**.

Principle 7: Low Coupling

- Components should have **minimal dependency** on each other.
- Low coupling reduces error propagation and improves maintainability.

Principle 8: Understandable Design

- Design models must be easy to understand for developers, testers, and maintainers.

Principle 9: Iterative Design

- Design evolves through **multiple iterations**.
- Each iteration aims to simplify and improve the design.

Principle 10: Design Supports Agile Development

- Design documentation is still important in agile projects.
- It helps in maintenance and understanding complex systems.
- Agile design documentation should stay **synchronized with code**.

Design Model Elements

1. Data Design Elements

- Focus on how data is structured, stored, and accessed.
 - Begins with a **high-level data model** and moves toward database and data structure design.
 - Strong data design improves performance and system quality.
-

2. Architectural Design Elements

- Provide an **overall view of the system**, similar to a house floor plan.
 - Derived from:
 1. Application domain knowledge
 2. Requirements model (use cases, analysis classes)
 3. Architectural styles and patterns
 - Represented as **interconnected subsystems**, each possibly having its own internal architecture.
-

3. Interface Design Elements

- Define how information flows:
 - Between user and system (UI)
 - Between system and external entities
 - Between internal components
 - Includes:
 - **User Interface (UI / UX)** design
 - **External interfaces** (devices, networks, other systems)
 - **Internal interfaces** between components
 - Interfaces must include **error handling and security features**.
-

4. Component-Level Design Elements

- Describe the **internal details of each software component**.
 - Include:
 - Local data structures
 - Algorithms
 - Interfaces to access component services
 - Represented using UML component diagrams, activity diagrams, pseudocode, etc.
-

5. Deployment-Level Design Elements

- Show how software components are deployed on **hardware environments**.
- Represented using **UML deployment diagrams**.
- Indicate:
 - Hardware nodes
 - Software subsystems allocated to each node
- Can be represented in:
 - **Descriptor form** (general hardware)
 - **Instance form** (specific hardware configurations)

▼ Software Design

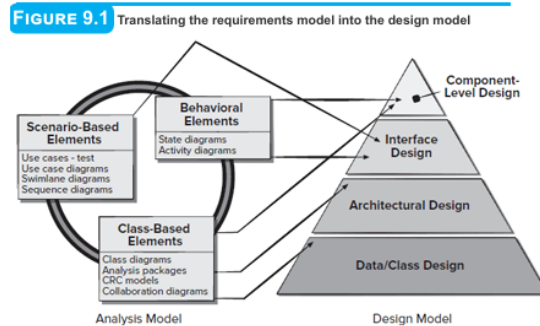
Role of Requirements Model in Design

The **requirements model** provides all the necessary information required to create a **complete design specification**. This model consists of:

- **Scenario-based elements** (use cases, user stories)
- **Class-based elements** (analysis classes, CRC models)
- **Behavioral elements** (state diagrams, sequence diagrams)

These elements collectively **feed the design process**. The transformation of requirements into design is illustrated in **Figure 9.1**, where the requirements

model is systematically translated into the **design model** using appropriate **design notations and methods**.



Four Design Models Produced from Requirements

A complete software design specification consists of **four interrelated design models**:

1. Data / Class Design

The **data/class design** transforms the **analysis class models** into **design class realizations** and defines the **data structures** required for implementation.

- Uses **CRC models**, class attributes, and object relationships
- Defines how data is **organized, stored, and accessed**
- Part of class design may occur alongside architectural design
- Detailed class design is finalized during **component design**

This design ensures that data handling is efficient, consistent, and aligned with system requirements.

2. Architectural Design

The **architectural design** defines the **overall structure of the software system**.

It specifies:

- Relationships between **major structural elements**
- The **architectural style** (layered, client-server, microservices, etc.)
- Design **patterns** used to meet system requirements
- Constraints affecting implementation (performance, scalability, security)

The architectural representation acts as the **framework of the system** and is derived directly from the **requirements model**.

3. Interface Design

The **interface design** describes how the software interacts with:

- **External systems**
- **Other software components**
- **Human users**

An interface implies:

- A **flow of information** (data/control)
- A **defined behavior**

Hence, **usage scenarios** and **behavioral models** are the primary inputs for interface design. Good interface design ensures usability, correctness, and smooth communication.

4. Component-Level Design

The **component-level design** converts architectural elements into **procedural descriptions of software components**.

- Focuses on internal logic of components
- Uses information from **class-based** and **behavioral models**
- Defines algorithms, control flow, and method logic

This design directly supports **coding and unit testing**.

Importance of Software Design

The importance of software design can be summarized in **one word: QUALITY**.

Software design:

- Is the **only phase** where quality can be **systematically built in**
- Provides representations that can be **evaluated before coding**
- Accurately translates **stakeholder requirements** into a working system
- Serves as the **foundation** for all later activities

Without proper design:

- Systems become **unstable**
- Small changes cause **major failures**
- Testing becomes difficult
- Quality issues are discovered **late**, when time and budget are limited

Late detection of problems is costly because **most resources have already been spent**.

▼ Architectural Styles

Definition of Architectural Style

An architectural style describes a **category of software systems** and defines:

1. **Components**

These are the major processing units of the system, such as databases, computational modules, servers, or user interface modules.

2. **Connectors**

These define how components communicate, coordinate, and cooperate with each other (e.g., method calls, message passing, pipes, or network protocols).

3. **Constraints**

Rules that restrict how components can be combined or integrated to form the system.

4. **Semantic models**

These help designers understand the **overall behavior and properties** of the system by examining the known behavior of individual components.

An architectural style is applied to the **entire system**, shaping its overall organization and structure.

Differences Between Architectural Style and Pattern

An architectural pattern differs from an architectural style in the following ways:

1. Limited Scope

A pattern focuses on **one specific aspect** of the architecture rather than the entire system.

2. Rule-Based Guidance

A pattern defines **rules** for handling a particular concern at the infrastructure level, such as concurrency, communication, or resource management.

3. Behavior-Oriented

Architectural patterns often address **specific behavioral problems**, such as:

- Synchronization in real-time systems
- Handling interrupts
- Managing concurrent execution.

1. Data-Centered Architecture

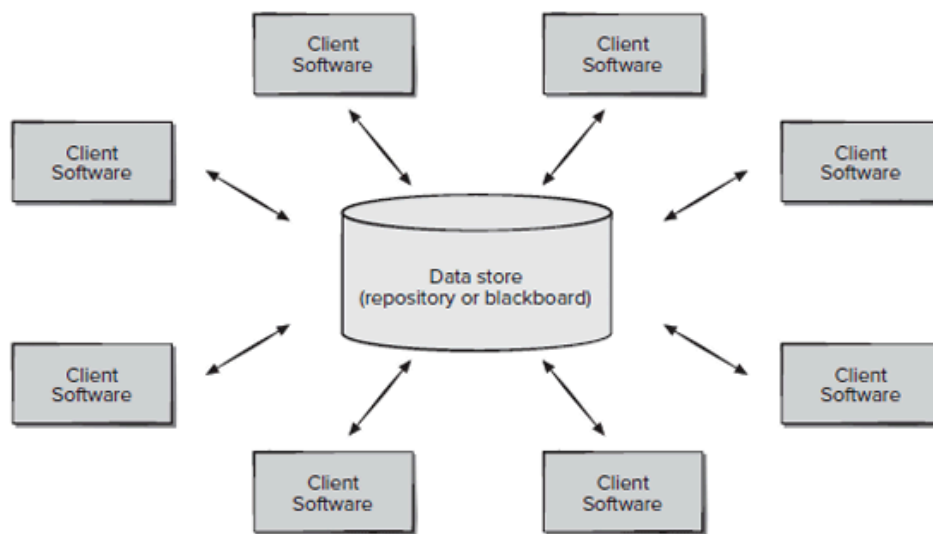
In a **data-centered architecture**, a **central data store** (such as a database or file system) resides at the core of the system. Other components, called **clients**, access this repository to **add, modify, delete, or retrieve data**.

- The data store may be **passive**, where clients access data independently.
- A variation is the **blackboard architecture**, where the repository actively **notifies clients** when relevant data changes.

Advantages:

- Promotes **integrability**
- New components can be added or modified without affecting others
- Supports coordination among clients via shared data

Example: Database-centric business applications



2. Data-Flow Architecture

This style is used when **input data must be transformed into output data** through a series of processing steps.

- It commonly uses the **Pipe-and-Filter pattern**
- Each processing unit is called a **filter**
- Filters are connected by **pipes**, which transmit data

Key Characteristics:

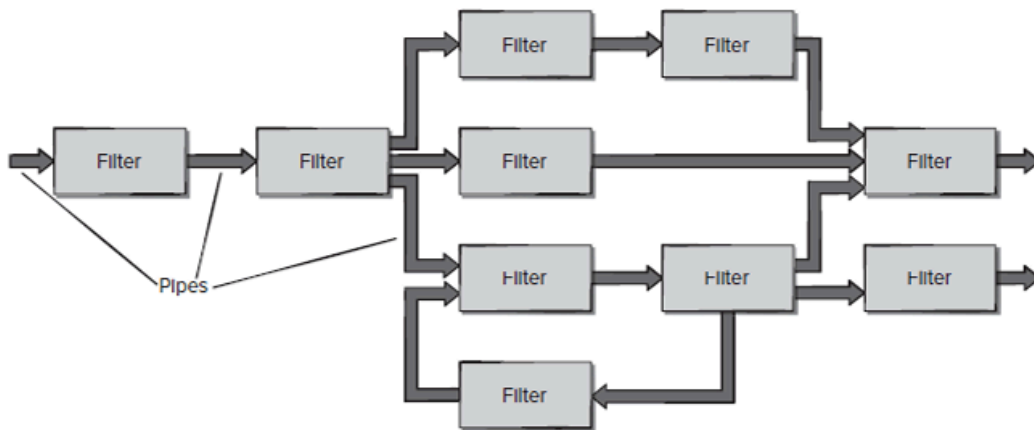
- Each filter operates **independently**
- Filters do not need to know the internal workings of other filters

- Data flows sequentially from one filter to another

Advantages:

- Easy to understand and modify
- High reusability of filters

Example: Compilers, signal processing systems



3. Call-and-Return Architecture

This architectural style organizes the system as a **control hierarchy** where control flows from a main component to subordinate components.

a) Main Program / Subprogram Architecture

- A **main program** controls execution
- Subprograms perform specific tasks
- Subprograms may call other subprograms

Advantages:

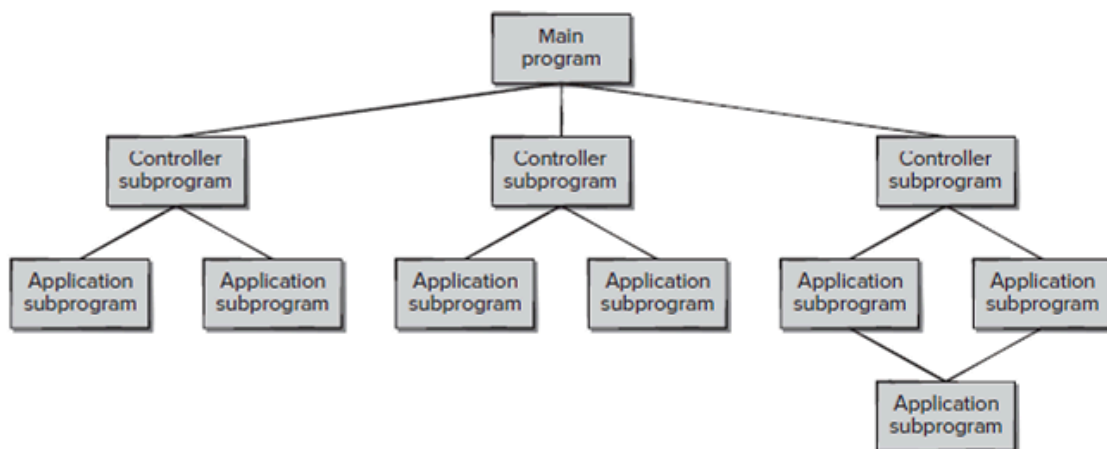
- Simple and easy to implement
- Well-structured control flow

b) Remote Procedure Call (RPC) Architecture

- Extends the main/subprogram style across **networked systems**
- Components are distributed across multiple machines
- Remote calls appear like local procedure calls

Example: Distributed client-server applications

FIGURE 10.3 Main program/subprogram architecture



4. Object-Oriented Architecture

In an **object-oriented architecture**, the system is organized as a collection of **objects**.

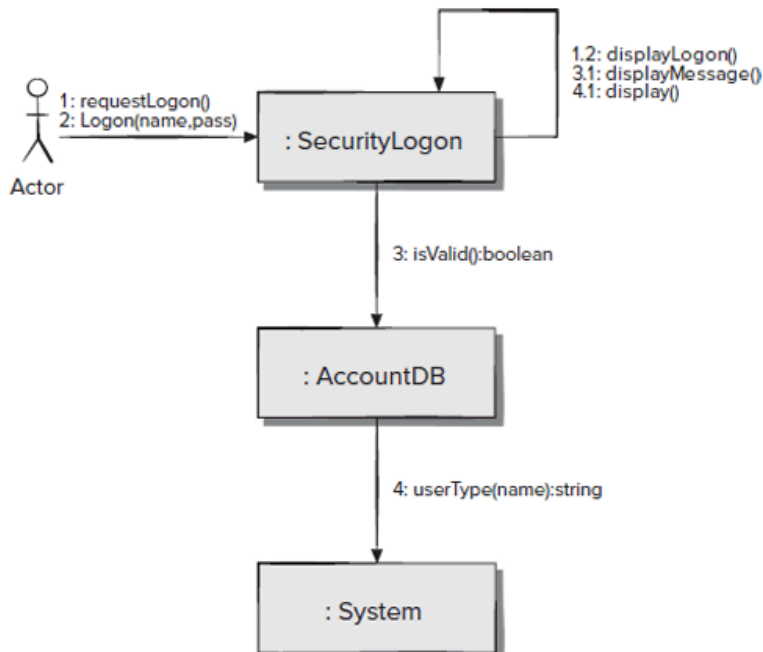
- Each object encapsulates **data and the operations** that manipulate it
- Objects communicate using **message passing**

Key Characteristics:

- High cohesion within objects
- Loose coupling between objects
- Supports reuse and maintainability

Example: Object-oriented enterprise applications

FIGURE 10.4 UML communication diagram



5. Layered Architecture

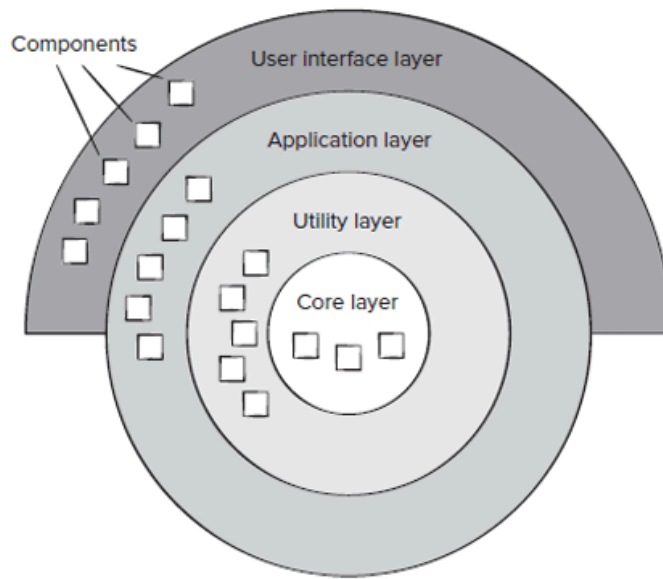
A **layered architecture** organizes the system into **hierarchical layers**, each with a specific responsibility.

- **Outer layers** handle user interaction
- **Middle layers** provide application and utility services
- **Inner layers** interact with the operating system and hardware

Advantages:

- Clear separation of concerns
- Easier testing and maintenance
- Changes in one layer have minimal impact on others

Example: Operating systems, network protocols



6. Model–View–Controller (MVC) Architecture

MVC is a specialized architectural style commonly used in **web and mobile applications**.

It divides the system into three components:

- **Model:** Contains application data and business logic
- **View:** Handles presentation and user interface
- **Controller:** Manages user input and coordinates interaction between model and view

Working:

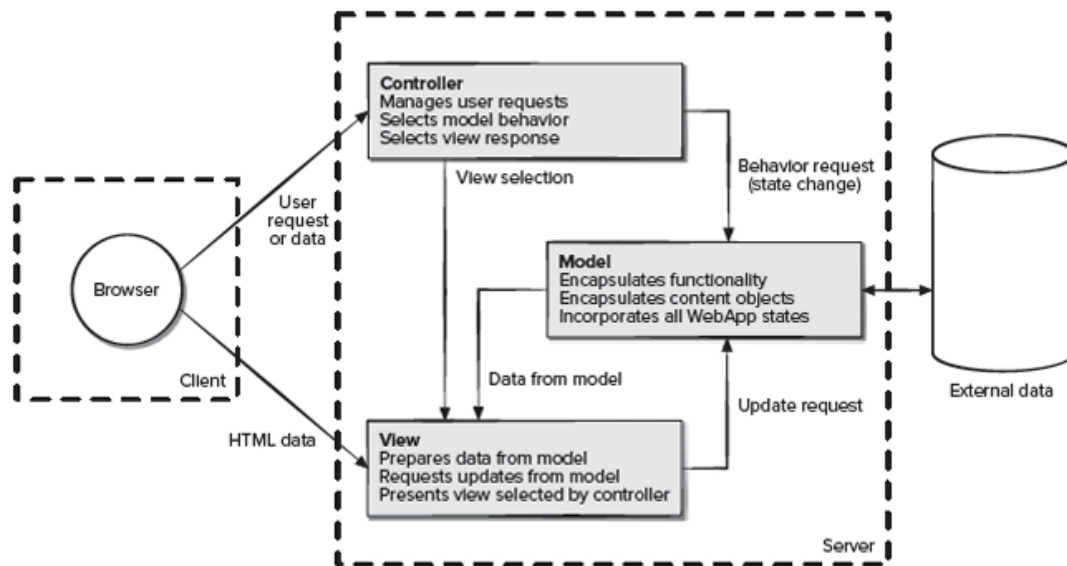
- User requests are handled by the controller
- The controller invokes operations on the model
- The model updates data and returns results
- The view formats and displays data to the user

Advantages:

- Clear separation of concerns

- Supports parallel development
- Improves maintainability
-

FIGURE 10.6 The MVC architecture



▼ Architectural Design

Architectural Archetypes

An **architectural archetype** is an abstraction (similar to a class) that represents a **core element of system behavior**.

Archetypes:

- Capture **stable and fundamental concepts** of the system
- Form the **basis of the system architecture**
- Do **not contain implementation details**

A small set of archetypes is usually sufficient, even for complex systems.

They are often derived from **analysis classes** identified during requirements analysis.

As design progresses, archetypes are **refined into concrete software components**.

(ACD) Context → Archetypes → Components → Detailed Design.

Draw ACD (from notes)

▼ Component Level Design

Component-Level Design – Required Theory

Component-Level Design

Component-level design uses information from:

- The **requirements model**
- The **architectural model**

In object-oriented software engineering, component-level design focuses on:

- Refining **problem-domain classes**
- Defining **infrastructure classes**
- Specifying **attributes, operations, and interfaces**

This detailed design serves as a **precursor to coding**.

Basic Design Principles (Component-Level)

The goal of these principles is to:

- Make designs **easy to change**
- Reduce **side effects** when changes occur

1. Open–Closed Principle (OCP)

A component should be:

- **Open for extension**
- **Closed for modification**

This means new behavior should be added **without changing existing code**, usually by introducing **abstractions**.

2. Liskov Substitution Principle (LSP)

Subclasses must be **substitutable** for their base classes.

Any derived class must:

- Honor the **preconditions**
- Preserve the **postconditions**

This ensures correct behavior when a base class reference is replaced by a derived class.

3. Dependency Inversion Principle (DIP)

High-level components should:

- Depend on **abstractions**
- Not depend on **concrete implementations**

This improves **flexibility** and makes systems easier to extend and maintain.

4. Interface Segregation Principle (ISP)

It is better to have:

- **Multiple client-specific interfaces**
- Rather than one large general-purpose interface

Each client should only depend on the operations it actually uses.

Packaging Principles (Component Organization)

Components are often grouped into **packages or subsystems**. The following principles guide proper packaging.

5. Reuse/Release Equivalency Principle (REP)

The unit of reuse should be the **unit of release**.

Reusable components should be:

- Packaged together
- Released and versioned together

This supports effective **reuse and configuration control**.

6. Common Closure Principle (CCP)

Classes that:

- **Change together**
- Should be **packaged together**

This improves change management and minimizes the impact of modifications.

7. Common Reuse Principle (CRP)

Classes that are:

- **Not reused together**
- Should **not be grouped together**

This avoids unnecessary recompilation, integration, and testing.

▼ Component-Level Design Guidelines

Component-Level Design Guidelines

In addition to basic design principles, **pragmatic guidelines** are applied during component-level design. These guidelines focus on:

- Components
- Interfaces
- Dependencies and inheritance

They help improve **clarity, maintainability, and readability** of the design.

1. Guidelines for Components

- **Naming conventions** should be clearly defined.
 - Architectural component names should:
 - Be derived from the **problem domain**
 - Be meaningful to all stakeholders
 - Infrastructure or implementation-specific components should:
 - Reflect **technical or implementation intent**
 - **Stereotypes** may be used to classify components, such as:
 - `<<infrastructure>>`
 - `<<database>>`
 - `<<table>>`
-

2. Guidelines for Interfaces

Interfaces describe **communication and collaboration** between components and support the **Open–Closed Principle**.

To reduce diagram complexity:

- Use **lollipop notation** instead of full UML interface boxes when diagrams become complex
- Interfaces should consistently flow from the **left-hand side** of the component
- Show **only relevant interfaces**, even if others exist

These practices improve **visual simplicity**.

3. Guidelines for Dependencies and Inheritance

- Model **dependencies** from **left to right**
- Model **inheritance** from **bottom (derived)** to **top (base)**
- Prefer **interface-based dependencies** rather than direct component-to-component dependencies

This improves **maintainability** and aligns with the **Open–Closed Principle**.

▼ Coupling , Cohesion and Functional Independence

1. Coupling

Definition:

- Coupling refers to the **degree of interdependence** between software modules.
- It measures how much one module relies on others.

Types of Coupling:

- **Content Coupling:**

One module directly modifies or relies on the internal workings of another. (Very bad)

- **Common Coupling:**

Modules share the same global data.

Modules share the same global state.

- **Control Coupling:**

One module controls the flow of another by passing control flags.

- **Stamp Coupling:**

Modules share a composite data structure but use only parts of it.

- **Data Coupling:**

Modules communicate by passing only the necessary data.
(Good)

- **Message Coupling:**

Modules interact only via message passing or interfaces.
(Best)

2. Cohesion

Definition:

Cohesion refers to the **degree to which elements inside a module belong together** — how focused a module is on a single task or purpose.

Types of Cohesion (from worst to best):

- **Coincidental Cohesion:**

Elements have no meaningful relationship. (Bad)

- **Logical Cohesion:**

Elements perform similar activities but are unrelated logically.

- **Temporal Cohesion:**

Elements are related by time (e.g., initialization tasks).

- **Procedural Cohesion:**

Elements follow a specific sequence of execution.

- **Communicational Cohesion:**

Elements operate on the same data.

- **Functional Cohesion:**

All elements contribute to a single well-defined task. (Best)

3. Functional Independence

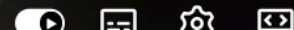
Definition:

Functional Independence is the principle that software modules should be **self-contained, with high cohesion and low coupling**, meaning they can operate independently without relying heavily on other modules.

Significance:

Achieving functional independence means modules can be developed, tested, maintained, and reused **individually with minimal impact on** others.

8:45



Component-Level Design Process

Step 1: Identify design classes in the problem domain.

In this step, all design classes that directly represent the problem domain are identified. These classes are derived from the analysis classes and architectural components defined during earlier phases. Each analysis class is elaborated to include more implementation-level details so that it can be directly used during coding.

Step 2: Identify design classes in the infrastructure domain.

This step focuses on identifying classes that support the system but are not part of the problem domain. These classes are usually not described in the requirements model and may be missing from the architecture model. Examples include graphical user interface components, operating system interfaces, and data or object management classes required for system operation.

Step 3: Elaborate all non-reusable design classes.

All design classes that are not acquired as reusable components are fully elaborated in this step. The interfaces, attributes, and operations of each class are described in sufficient detail to allow implementation. During elaboration, design heuristics such as high cohesion and low coupling are carefully considered to improve software quality.

Step 4: Describe persistent data sources.

In this step, persistent data sources such as databases and files are identified and described. The structure and organization of these data stores are refined, and the classes responsible for managing persistent data are defined. This ensures proper data storage and retrieval across the system.

Step 5: Develop behavioral representations.

Behavioral models are developed to describe how a design class behaves over time in response to events. UML state diagrams are used to represent states, transitions, events, guard conditions, and actions. These models help in understanding the dynamic behavior of objects during execution.

Step 6: Elaborate deployment diagrams.

Deployment diagrams are refined to provide additional implementation details. This step identifies the hardware and operating system environments and shows where major component packages are deployed. The goal is to clearly represent the physical configuration of the system.

Step 7: Refactor and evaluate alternatives.

Component-level design is iterative, so all design representations are reviewed and refactored to improve consistency, completeness, and accuracy. Alternative design solutions are considered, and the best option is selected using established design principles and heuristics.

▼ Defining Software

Software plays a dual role in modern systems: it acts as a **product** that delivers information by processing, managing, and transmitting data, and as a **vehicle** that enables computer control through operating systems, communication via networks, and development through software tools. Unlike hardware, software is a **logical system** and does not wear out due to environmental factors.

Hardware follows a bathtub failure curve with early failures, a stable period, and eventual wear-out, whereas software ideally shows high initial failures due to bugs that reduce after correction. However, in reality, software undergoes frequent changes to add features or fix issues, and each change can introduce new errors, causing repeated spikes in failure rate. Over time, the minimum failure rate increases, leading to **software deterioration**, even though it does not physically wear out. Since software has no spare parts and every failure indicates a design or coding error, software maintenance is more complex than hardware maintenance, which has led to the need for disciplined **software engineering practices**.

▼ Software domain applications

System Software:

System software is a collection of programs designed to support and service other programs. It includes operating systems, compilers, editors, device drivers, and networking software. Some system software processes well-defined data structures, while others handle indeterminate data such as system events and user interactions, making their design and maintenance complex.

Application Software:

Application software consists of stand-alone programs developed to solve specific business or user problems. These programs process business or technical data to support operations, decision-making, and management activities, such as accounting systems, payroll software, and inventory management applications.

Engineering and Scientific Software:

Engineering and scientific software is used for numerical computation, data analysis, and simulation in scientific and engineering domains. It includes applications for areas such as computer-aided design, weather forecasting, genetic analysis, automotive stress testing, and space research, often requiring high precision and performance.

Embedded Software:

Embedded software resides within a product or system and is used to control and monitor its functions. It may perform simple tasks, such as keypad control in a microwave oven, or complex operations, such as fuel control, braking systems, and dashboard displays in modern automobiles.

Product-Line Software:

Product-line software is developed using reusable components to provide specific capabilities to a wide range of customers. It may target a niche market, such as inventory control systems, or a mass consumer market, enabling faster development and consistency across products.

Web and Mobile Applications:

Web and mobile applications are network-centric software systems that include browser-based applications, cloud computing platforms, service-oriented systems, and mobile device applications. These applications

emphasize connectivity, scalability, and user accessibility across different devices.

Artificial Intelligence Software:

Artificial intelligence software uses heuristics and learning techniques to solve complex problems that cannot be addressed by traditional algorithms.

Applications include robotics, machine learning, image and speech recognition, decision-support systems, theorem proving, and game playing.

▼ Defining the discipline

Software Engineering Definition:

According to IEEE, software engineering is the application of a **systematic, disciplined, and quantifiable approach** to the development, operation, and maintenance of software, essentially applying engineering principles to software development. However, the level of discipline required may vary between teams, so along with structure, adaptability and agility are also essential.

Layered Technology of Software Engineering:

Software engineering is described as a **layered technology**. At the base lies an **organizational commitment to quality**, inspired by philosophies such as Total Quality Management (TQM) and Six Sigma, which promote continuous process improvement. This quality focus forms the bedrock of effective software engineering.

Process Layer:

The **process layer** acts as the foundation and glue that holds all other layers together. It provides a structured framework for planning, managing, and controlling software projects. The software process defines activities, milestones, work products, quality assurance practices, and change management, ensuring timely and systematic software development.

Methods Layer:

Software engineering **methods** provide the technical “how-to” for building software. They include activities such as communication, requirements analysis, design, coding, testing, and maintenance. These methods are based

on established principles and use models and documentation to describe and understand the system.

Tools Layer:

Software engineering **tools** offer automated or semi-automated support to both process and methods. When multiple tools are integrated to share information seamlessly, they form a **Computer-Aided Software Engineering (CASE)** environment, which improves productivity and consistency in software development

▼ Process Framework

Process Framework

A **process framework** provides the foundation for a complete software engineering process by defining a small set of framework activities that apply to all software projects, irrespective of their size or complexity. Along with these, it includes **umbrella activities** that span the entire process. The generic software process framework consists of five core activities: communication, planning, modeling, construction, and deployment.

Communication

The **communication** activity focuses on interaction and collaboration with customers and other stakeholders before technical work begins. Its goal is to understand stakeholder objectives and gather requirements that define the software's features and functions.

Planning

Planning establishes a roadmap for the software project. It results in a software project plan that defines technical tasks, identifies risks, estimates required resources, specifies work products, and sets a project schedule to guide development.

Modeling

The **modeling** activity involves creating representations of software requirements and design to understand the problem and its solution. Models help visualize system structure, behavior, and relationships, and are refined progressively to achieve clarity and accuracy.

Construction

The **construction** activity combines code generation and testing. It focuses on building the software according to the design and uncovering errors through systematic testing to ensure correctness and quality.

Deployment

Deployment involves delivering the software, either fully or as an incremental version, to the customer. The customer evaluates the product and provides feedback, which is used to improve subsequent versions.

Iterative Nature of the Process

These five framework activities are applied to all types of software projects and are often performed **iteratively**. Each iteration produces a software increment with added features and functionality, gradually making the software more complete and refined

▼ Software engineering practice

The essence of software engineering practice is based on George Polya's problem-solving approach, which emphasizes four fundamental steps: understanding the problem, planning a solution, carrying out the plan, and examining the result. Understanding the problem involves identifying stakeholders, clarifying requirements, recognizing unknowns, decomposing the problem into smaller parts, and representing it using models. Planning the solution focuses on design activities such as recognizing similar problems, reusing existing solutions, defining subproblems, and creating design models. Carrying out the plan corresponds to implementation, where code is developed according to the design and reviewed to ensure correctness and traceability. Examining the result involves testing and validation to ensure the software meets all functional requirements and stakeholder expectations. This

structured yet commonsense approach ensures quality, correctness, and reliability in software development.